claude-windows / 6cee04e1-f4f1-4593-86c9-2e3ca3473efd

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\subagents\workflows\wf_e5140909-890\agent-a794cd1d1248b7158.jsonl`
- SHA-256: `fc97c3ddab10d9dc930e9fd49cc5e9de67a2787951e563606058b3978e047fe8`
- Source modified: `2026-06-18T07:52:16+00:00`
- Imported at: `2026-07-05T16:48:26+00:00`
- project: `wf_e5140909-890`
- session_id: `6cee04e1-f4f1-4593-86c9-2e3ca3473efd`

Transcript

1. user (2026-06-18T07:47:49.192Z)

```
Work READ-ONLY in this git worktree pinned to origin/master (DO NOT modify):
  C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/refactor-base
Read the ACTUAL code (and the matching tests/ file) before asserting anything; never guess.
Adversarial bug-hunter. Report ONLY real, reachable defects (crash on a reachable input, wrong
logic, inverted condition, bad error handling, resource leak, off-by-one, contract mismatch,
unguarded external/persisted-data access). Exact file+line + concrete trigger + minimal fix +
confidence. PRECISION over recall ? empty list is a respected answer; do NOT invent findings.
The repo has ~855 passing tests + several hardening sweeps; expect few. Skip pure style.
AREA: backend/pipeline/segmenters/ + stitcher/ + validators/. The big process_video methods,
compiler.compile_highlights padding/merge math, validators. Unguarded Result/segment access,
subprocess returncode, padding/overlap math.
```

2. assistant (2026-06-18T07:47:51.584Z)

```
I'll analyze the segmenters, stitcher, and validators code for real defects. Let me start by
mapping the relevant files.
```

3. user (2026-06-18T07:47:52.155Z)

```
backend\pipeline\segmenters\__init__.py
backend\pipeline\segmenters\base.py
backend\pipeline\segmenters\fusion_hybrid.py
backend\pipeline\segmenters\gemini.py
backend\pipeline\segmenters\motion.py
backend\pipeline\segmenters\tracknet_hybrid.py
backend\pipeline\segmenters\trajectory_hybrid.py
backend\pipeline\segmenters\wasb_hybrid.py
backend\pipeline\segmenters\yolo_hybrid.py
```

4. user (2026-06-18T07:47:52.642Z)

```
backend\pipeline\stitcher\compiler.py
```

5. user (2026-06-18T07:47:52.688Z)

```
backend\pipeline\validators\base.py
backend\pipeline\validators\__init__.py
backend\pipeline\validators\blur.py
backend\pipeline\validators\format.py
backend\pipeline\validators\pts_continuity.py
backend\pipeline\validators\relevance.py
backend\pipeline\validators\resolution_fps.py
backend\pipeline\validators\scene_cut.py
```

6. assistant (2026-06-18T07:47:54.528Z)

Let me read the core files. Starting with the stitcher/compiler (padding/merge math) and the base classes.

7. user (2026-06-18T07:47:55.480Z)

```
1       import os
2       import subprocess
3       import tempfile
4       import logging
5       from pathlib import Path
6       from typing import Any, Dict, List, Optional, Tuple, Union
7
8       logger = logging.getLogger(__name__)
9
10      # Bundled, commercially-clean default watermark font (Apache-2.0). Shipped so the
overlay
11      # renders out-of-the-box without a system fontconfig setup (a common gap on Windows,
where
12      # `font='Sans'` fails). Overridable via stitching.watermark.font_path. This module lives
at
13      # backend/pipeline/stitcher/, so parents[2] == backend/.
14      _BUNDLED_FONT = Path(__file__).resolve().parents[2] / "assets" / "fonts" / "RobotoSlab-
Regular.ttf"
15
16      class VideoCompiler:
17          def __init__(self, output_dir: str, end_padding: float = 1.0, begin_padding: float =
0.5,
18                       watermark: Optional[Any] = None):
19              self.output_dir = output_dir
20              self.end_padding = end_padding
21              self.begin_padding = begin_padding
22              # Optional branding overlay. Accepts a WatermarkConfig model or a plain dict;
23              # normalized to a dict (or None) so this module stays config-package-agnostic.
24              self.watermark = self._normalize_watermark(watermark)
25              os.makedirs(output_dir, exist_ok=True)
26
27          @staticmethod
28          def _normalize_watermark(watermark: Optional[Any]) -> Optional[Dict[str, Any]]:
29              if watermark is None:
30                  return None
31              if hasattr(watermark, "model_dump"):
32                  return watermark.model_dump()
33              if isinstance(watermark, dict):
34                  return dict(watermark)
35              return None
36
37          @staticmethod
38          def _ff_quote(path: str) -> str:
39              """Make a filesystem path safe to embed inside single quotes in an ffmpeg
40              filtergraph operand (drawtext fontfile/textfile, movie= source): forward
41              slashes (Windows-friendly), escaped single quotes, AND escaped colons.
42
43              The colon MUST be escaped even inside single quotes: ffmpeg's filtergraph
44              option parser treats ':' as the option separator, so a Windows drive-letter
45              path like ``C:/dir/font.ttf`` otherwise truncates the operand at ``C`` and
46              the encode fails with "Invalid argument". This silently disabled the
47              on-by-default bundled-font watermark (and absolute logo paths) on Windows."""
48              return path.replace("\\", "/").replace("'", r"\'").replace(":", r"\:")
49
50          @staticmethod
51          def _parse_time(val: Union[int, float, str]) -> float:
52              """Normalizes a timestamp value to float seconds.
53              Handles: float/int (pass-through), plain numeric strings ("12.4"),
```

```
54              and colon-separated strings ("MM:SS", "HH:MM:SS", "H:MM:SS.ms").
55              This mirrors the parse_time logic used in gemini.py to ensure
56              the stitcher can always consume whatever format the indexer produced."""
57              if isinstance(val, (int, float)):
58                  return float(val)
59              if isinstance(val, str):
60                  val = val.strip()
61                  if ":" in val:
62                      # Handle MM:SS, HH:MM:SS, etc.
63                      parts = val.split(":")
64                      parts.reverse()
65                      total = 0.0
66                      for i, p in enumerate(parts):
67                          try:
68                              total += float(p) * (60 ** i)
69                          except ValueError:
70                              pass
71                      return total
72                  try:
73                      return float(val)
74                  except ValueError:
75                      logger.warning(f"Could not parse timestamp value '{val}' as a number.
Defaulting to 0.0.")
76                      return 0.0
77              logger.warning(f"Unexpected timestamp type {type(val).__name__} for value
'{val}'. Defaulting to 0.0.")
78              return 0.0
79
80          @staticmethod
81          def _merge_overlapping(segments: List[Tuple[float, float]]) -> List[Tuple[float,
float]]:
82              """Normalize, sort, and merge overlapping/duplicate (start, end) ranges.
83
84              A highlight reel must never replay the same footage. The same rally can land in
85              the segment list more than once ? e.g. two detector runs accumulated for one
86              video (add_video is an UPSERT, so re-running without a prune leaves both sets),
or
87              chunk-overlap duplicates near boundaries. Stitching those verbatim plays the
rally
88              twice. Merging any range that overlaps or abuts the previous one collapses true
89              duplicates/overlaps while preserving genuinely distinct (gapped) rallies.
90              Degenerate rows (end <= start) are dropped. Timestamps may be float/int/str.
91              """
92              parsed = []
93              for raw_s, raw_e in segments:
94                  s = VideoCompiler._parse_time(raw_s)
95                  e = VideoCompiler._parse_time(raw_e)
96                  if e > s:
97                      parsed.append((s, e))
98              parsed.sort()
99              merged: List[Tuple[float, float]] = []
100             for s, e in parsed:
101                 if merged and s <= merged[-1][1]:   # overlaps/abuts previous -> extend it
102                     merged[-1] = (merged[-1][0], max(merged[-1][1], e))
103                 else:
104                     merged.append((s, e))
105             return merged
106
107         def _get_video_duration(self, video_path: str) -> float:
108             """Probes the video file to get its total duration in seconds.
109             Returns 0.0 if it cannot be determined."""
110             try:
111                 result = subprocess.run(
112                     [
113                         "ffprobe", "-v", "error",
```

```
                        "-show_entries", "format=duration",
                        "-of", "default=noprint_wrappers=1:nokey=1",
                        video_path
                    ],
                    capture_output=True, text=True, check=True
                )
                return float(result.stdout.strip())
            except Exception as e:
                logger.warning(f"Could not determine video duration via ffprobe: {e}")
                return 0.0

        def _watermark_filter(self, tmp_dir: str) -> Optional[str]:
            """Build the ffmpeg -vf filtergraph that burns the configured watermark (text
            and/or logo PNG) into a clip during the per-clip re-encode. Returns None when
            watermarking is disabled or has nothing to draw. The concat stage stays -c copy
            because the mark is already baked into every clip identically."""
            wm = self.watermark
            if not wm or not wm.get("enabled"):
                return None

            text = (wm.get("text") or "").strip()
            logo_path = (wm.get("logo_path") or "").strip()
            if logo_path and not os.path.exists(logo_path):
                logger.warning(f"[watermark] logo not found: {logo_path} ? skipping logo
overlay.")
                logo_path = ""
            if not text and not logo_path:
                return None

            margin = int(wm.get("margin", 24))
            opacity = float(wm.get("opacity", 0.85))
            position = str(wm.get("position", "bottom-right"))
            # drawtext uses lowercase w/h (main) + tw/th (text); overlay uses W/H (main) +
w/h (logo).
            dt_xy = {
                "bottom-right": f"x=w-tw-{margin}:y=h-th-{margin}",
                "bottom-left": f"x={margin}:y=h-th-{margin}",
                "top-right": f"x=w-tw-{margin}:y={margin}",
                "top-left": f"x={margin}:y={margin}",
            }.get(position, f"x=w-tw-{margin}:y=h-th-{margin}")
            ov_xy = {
                "bottom-right": f"W-w-{margin}:H-h-{margin}",
                "bottom-left": f"{margin}:H-h-{margin}",
                "top-right": f"W-w-{margin}:{margin}",
                "top-left": f"{margin}:{margin}",
            }.get(position, f"W-w-{margin}:H-h-{margin}")

            drawtext = ""
            if text:
                ratio = float(wm.get("font_size_ratio") or 0.035)
                divisor = max(1, round(1.0 / ratio))
                # textfile= sidesteps the notorious drawtext text= escaping for arbitrary
strings.
                tf_path = os.path.join(tmp_dir, "watermark.txt")
                with open(tf_path, "w", encoding="utf-8") as fh:
                    fh.write(text)
                # Font precedence: explicit font_path ? bundled Apache-2.0 default (renders
                # without fontconfig) ? fontconfig family (last-resort, environment-
dependent).
                font_path = (wm.get("font_path") or "").strip()
                if not font_path and _BUNDLED_FONT.exists():
                    font_path = str(_BUNDLED_FONT)
                if font_path:
                    font_opt = f"fontfile='{self._ff_quote(font_path)}'"
                else:
```

```
                    font_opt = f"font='{wm.get('font', 'Sans')}'"
                box = ""
                if wm.get("box", True):
                    box = f":box=1:boxcolor=black@{min(opacity, 0.5):.2f}:boxborderw=10"
                drawtext = (
                    f"drawtext=textfile='{self._ff_quote(tf_path)}':{font_opt}"
                    f":fontcolor=white@{opacity:.2f}:fontsize=h/{divisor}:{dt_xy}{box}"
                )

            if logo_path and drawtext:
                return
f"movie='{self._ff_quote(logo_path)}'[wm];[in][wm]overlay={ov_xy},{drawtext}"
            if logo_path:
                return f"movie='{self._ff_quote(logo_path)}'[wm];[in][wm]overlay={ov_xy}"
            return drawtext

        def compile_highlights(self, raw_video_path: str, segments: List[Tuple[float,
float]], output_filename: str) -> str:
            """
            Extracts selected segments from a raw video and stitches them together
            into a seamless highlights file, applying end_padding to prevent abrupt endings.
            """
            if not segments:
                raise ValueError("No highlight segments provided to compile.")

            # Never stitch the same footage twice: collapse overlapping/duplicate ranges
            # (e.g. accumulated re-run segments or chunk-overlap dupes) before trimming.
            original_count = len(segments)
            segments = self._merge_overlapping(segments)
            if len(segments) < original_count:
                logger.info("Merged %d overlapping/duplicate segment(s) before stitching: %d
-> %d.",
                            original_count - len(segments), original_count, len(segments))
            if not segments:
                raise ValueError("No valid (start < end) highlight segments after merge.")

            if not os.path.exists(raw_video_path):
                raise FileNotFoundError(f"[COMPILER ERROR] Source video file not found:
{raw_video_path}")

            # Defense-in-depth: never let the output name escape output_dir (the API also
            # validates this at the request boundary).
            output_path = os.path.join(self.output_dir, os.path.basename(output_filename))

            # Probe video duration so we can detect out-of-range segments
            video_duration = self._get_video_duration(raw_video_path)
            if video_duration > 0:
                logger.info(f"Source video duration: {video_duration:.2f}s")
            else:
                logger.warning("Video duration unknown. Cannot validate segment boundaries
against video length.")

            # We will create temporary clips and stitch them using FFmpeg concat demuxer
            temp_clips = []
            skipped_segments = []

            # Create a temp directory within output_dir
            with tempfile.TemporaryDirectory(dir=self.output_dir) as tmp_dir:
                logger.info(f"Trimming {len(segments)} segments
(begin_padding={self.begin_padding}s, end_padding={self.end_padding}s)...")
                # Build the (optional) watermark filtergraph once; it is applied during
every
                # per-clip re-encode so the mark is baked into each clip identically.
                watermark_filter = self._watermark_filter(tmp_dir)
                if watermark_filter:
```

```
                    logger.info("[watermark] applying branding overlay to each clip.")
                for idx, (raw_start, raw_end) in enumerate(segments):
                    # Normalize timestamps to float seconds ? handles float, int,
                    # plain numeric strings, and colon-separated formats like "MM:SS"
                    start = self._parse_time(raw_start)
                    end = self._parse_time(raw_end)
                    segment_label = f"Segment #{idx+1}/{len(segments)} [{start:.2f}s -
{end:.2f}s]"

                    # Validate the segment times
                    if start < 0:
                        logger.warning(f"{segment_label}: start_time is negative
({start:.2f}s). Clamping to 0.")
                        start = 0.0

                    if start >= end:
                        logger.error(f"{segment_label}: start_time ({start:.2f}s) >=
end_time ({end:.2f}s). Skipping invalid segment.")
                        skipped_segments.append((idx, start, end, "start >= end"))
                        continue

                    # Check if segment extends beyond video duration
                    if video_duration > 0:
                        if start >= video_duration:
                            logger.error(f"{segment_label}: start_time ({start:.2f}s) is
beyond the video duration ({video_duration:.2f}s). "
                                         f"This segment cannot be extracted ? the video ran
out. Skipping.")
                            skipped_segments.append((idx, start, end, "start beyond video
end"))
                            continue

                        if end > video_duration:
                            original_end = end
                            end = video_duration
                            logger.warning(f"{segment_label}: end_time ({original_end:.2f}s)
exceeds video duration ({video_duration:.2f}s). "
                                           f"Clamping end_time to {video_duration:.2f}s. The
segment may be truncated.")

                    clip_path = os.path.join(tmp_dir, f"clip_{idx}.mp4")

                    # Precise sub-second trim using fast re-encoding to preserve stream
headers
                    padded_start = max(0.0, start - self.begin_padding)
                    padded_end = end + self.end_padding

                    # Clamp padded_end to video duration if known
                    if video_duration > 0 and padded_end > video_duration:
                        padded_end = video_duration
                        logger.info(f"{segment_label}: Padded end ({end +
self.end_padding:.2f}s) exceeds video duration. "
                                    f"Clamping to {video_duration:.2f}s (end padding
partially applied).")

                    trim_duration = padded_end - padded_start
                    logger.info(f"Trimming {segment_label} -> [{padded_start:.3f}s -
{padded_end:.3f}s] (duration: {trim_duration:.2f}s)")

                    base_cmd = [
                        "ffmpeg", "-y",
                        "-ss", str(round(padded_start, 3)),
                        "-to", str(round(padded_end, 3)),
                        "-i", raw_video_path,
                        "-c:v", "libx264",
```

```
                        "-preset", "superfast",
                        "-crf", "22",
                        "-c:a", "aac",
                        "-b:a", "128k",
                    ]
                    vf_args = ["-vf", watermark_filter] if watermark_filter else []

                    # Try with the watermark first; if that encode fails (e.g. a bad
font/logo
                    # path), retry once WITHOUT it so a branding misconfig can never nuke
the
                    # whole reel. Clips stay codec-identical either way, so concat -c copy
holds.
                    attempts = [vf_args, []] if vf_args else [[]]
                    produced = False
                    last_exc: Optional[subprocess.CalledProcessError] = None
                    cmd = base_cmd + [clip_path]
                    for attempt_vf in attempts:
                        cmd = base_cmd + attempt_vf + [clip_path]
                        try:
                            subprocess.run(cmd, capture_output=True, check=True)
                        except subprocess.CalledProcessError as e:
                            last_exc = e
                            if attempt_vf and vf_args:
                                _err = (e.stderr.decode(errors="replace").strip()[-300:]
                                        if getattr(e, "stderr", None) else "")
                                logger.warning(
                                    f"{segment_label}: watermark encode failed; retrying
without it. "
                                    f"ffmpeg: {_err or '(no stderr captured)'}"
                                )
                                continue
                            break
                        if os.path.exists(clip_path) and os.path.getsize(clip_path) > 0:
                            produced = True
                            if not attempt_vf and vf_args:
                                logger.warning(f"{segment_label}: encoded WITHOUT the
watermark (the watermark filter failed).")
                            break
                        last_exc = None  # ffmpeg succeeded but produced nothing
                        break

                    if produced:
                        temp_clips.append(clip_path)
                    elif last_exc is not None:
                        stderr_msg = last_exc.stderr.decode(errors='replace').strip()
                        logger.error(f"{segment_label}: FFmpeg trim failed.")
                        print(f"  Command: {' '.join(cmd)}")
                        print(f"  FFmpeg stderr: {stderr_msg[-500:]}")  # Last 500 chars to
avoid flooding
                        skipped_segments.append((idx, start, end, f"ffmpeg error:
{stderr_msg[-200:]}"))
                        continue
                    else:
                        logger.error(f"{segment_label}: FFmpeg exited successfully but
output clip is missing or empty. Skipping.")
                        skipped_segments.append((idx, start, end, "empty output clip"))
                        continue

                # Report summary of skipped segments
                if skipped_segments:
                    logger.info(f"\n[SUMMARY] {len(skipped_segments)} out of {len(segments)}
segments were SKIPPED:")
                    for seg_idx, seg_start, seg_end, reason in skipped_segments:
                        logger.info(f"  - Segment #{seg_idx+1} [{seg_start:.2f}s -
```

```
{seg_end:.2f}s]: {reason}")
                    logger.info("")

                if not temp_clips:
                    raise RuntimeError(
                        f"[COMPILER ERROR] Failed to generate ANY valid highlight clips. "
                        f"All {len(segments)} segments were skipped. "
                        f"Check the errors above for details. "
                        f"Common causes: video file is corrupted, segments are out of range,
or ffmpeg is not installed."
                    )

                logger.info(f"Successfully trimmed {len(temp_clips)}/{len(segments)} clips.
Preparing to stitch...")

                # Create a list file for FFmpeg concat demuxer
                list_file_path = os.path.join(tmp_dir, "clips.txt")
                with open(list_file_path, "w") as f:
                    for clip in temp_clips:
                        # Escape single quotes in path
                        safe_path = clip.replace("'", "'\\''")
                        f.write(f"file '{safe_path}'\n")

                logger.info("Stitching clips together...")
                # Stitch them together without re-encoding (since they were encoded with
identical settings)
                concat_cmd = [
                    "ffmpeg", "-y",
                    "-f", "concat",
                    "-safe", "0",
                    "-i", list_file_path,
                    "-c", "copy",
                    output_path
                ]

                try:
                    subprocess.run(concat_cmd, capture_output=True, check=True)

                    # Final verification
                    if not os.path.exists(output_path) or os.path.getsize(output_path) == 0:
                        raise RuntimeError(
                            f"[COMPILER ERROR] FFmpeg concatenation appeared to succeed, "
                            f"but output file is missing or empty: {output_path}"
                        )

                    output_size_mb = os.path.getsize(output_path) / (1024 * 1024)
                    logger.info(f"Dynamic highlight reel saved: {output_path}
({output_size_mb:.1f} MB)")
                    return output_path
                except subprocess.CalledProcessError as e:
                    stderr_msg = e.stderr.decode(errors='replace').strip()
                    raise RuntimeError(
                        f"[COMPILER ERROR] FFmpeg concatenation failed.\n"
                        f"  Command: {' '.join(concat_cmd)}\n"
                        f"  FFmpeg stderr: {stderr_msg[-500:]}"
                    ) from e

```

8. user (2026-06-18T07:47:56.021Z)

```
1       from abc import ABC, abstractmethod
2       from typing import List, Dict, Any, Type, Optional
3       from backend.infrastructure.database import Database
4       from backend.results import Result  # new standardized contract (Result[Value] ==
Success[Value] | Failure)
```

```
5
6       class BasePlaySegmenter(ABC):
7           def __init__(self, db: Database, config: Dict[str, Any]):
8               self.db = db
9               self.config = config
10
11          @property
12          @abstractmethod
13          def name(self) -> str:
14              """Unique identifier for this segmenter."""
15              pass
16
17          @property
18          @abstractmethod
19          def description(self) -> str:
20              """Human-readable explanation of this segmenter."""
21              pass
22
23          @abstractmethod
24          def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None, max_frames: int = None) -> "Result[List[Dict[str, Any]]]":
25              """
26              Processes a video to detect play segments, populates SQLite database,
27              and returns a Result.
28
29              Use isinstance(result, Failure) (or result.is_success()) to distinguish
30              "no segments found" from "something went wrong" (missing key, healthcheck
31              failure, etc.). This is the start of the standardized error/result contract.
32              """
33              pass
34
35      class SegmenterRegistry:
36          def __init__(self):
37              self._segmenters: Dict[str, Type[BasePlaySegmenter]] = {}
38
39          def register(self, segmenter_cls: Type[BasePlaySegmenter]):
40              """Registers a new segmenter class under its name property."""
41              try:
42                  temp_inst = segmenter_cls(None, {})
43                  name = temp_inst.name
44              except Exception as e:
45                  raise ValueError(f"Failed to resolve name property from segmenter class
{segmenter_cls.__name__}: {e}") from e
46              self._segmenters[name] = segmenter_cls
47
48          def get(self, name: str) -> Optional[Type[BasePlaySegmenter]]:
49              return self._segmenters.get(name)
50
51          def list_available(self) -> Dict[str, str]:
52              """Returns a dict of registered segmenter names and their descriptions."""
53              res = {}
54              for name, cls in self._segmenters.items():
55                  try:
56                      temp_inst = cls(None, {})
57                      desc = temp_inst.description
58                  except Exception:
59                      desc = "No description available."
60                  res[name] = desc
61              return res
62
63      segmenter_registry = SegmenterRegistry()
64
```

9. assistant (2026-06-18T07:47:58.984Z)

Let me look at the larger segmenters. Starting with the hybrid ones (the big `process_video` methods).

10. user (2026-06-18T07:47:59.414Z)

```
1       import cv2
2       import time
3       import random
4       from typing import List, Dict, Any, Tuple
5       import logging
6       from backend.pipeline.segmenters.base import BasePlaySegmenter, segmenter_registry
7       from backend.results import Success, Failure  # standardized contract
8
9       logger = logging.getLogger(__name__)
10
11      class MotionPlaySegmenter(BasePlaySegmenter):
12          @property
13          def name(self) -> str:
14              return "motion"
15
16          @property
17          def description(self) -> str:
18              return "Traditional CV pixel-motion heuristics tracking frame differences."
19
20          def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None, max_frames: int = None) -> List[Dict[str, Any]]:
21              """
22              Processes a video to detect play segments using motion heuristics,
23              populates SQLite with detected segments, and indexes detailed events.
24              """
25              if not player_pool:
26                  player_pool = ["Avi", "Suresh", "Ramesh", "Deepak"]
27
28              cap = cv2.VideoCapture(video_path)
29              if not cap.isOpened():
30                  logger.error(f"Segmenter could not open video: {video_path}")
31                  return Failure(message=f"Could not open video: {video_path}",
is_recoverable=False)
32
33              fps = cap.get(cv2.CAP_PROP_FPS)
34              total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
35              duration = total_frames / fps if fps > 0 else 0
36
37              # Sub-sample settings (5 FPS for analysis)
38              analysis_fps = 5.0
39              frame_interval = max(1, int(fps / analysis_fps))
40
41              idx_cfg = self.config.get("indexing", {})
42              motion_threshold = idx_cfg.get("motion_threshold", 0.08)
43              min_segment_duration = idx_cfg.get("min_segment_duration", 3.0)
44              dead_time_merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
45
46              prev_gray = None
47              motion_signals = []
48              timestamps = []
49
50              frame_idx = 0
51              processed_count = 0
52              t_start = time.time()
53              # Emit a progress line roughly every 5% of the video so long runs are
54              # observable instead of silent (a full 1080p match is ~tens of minutes).
55              progress_step = max(1, total_frames // 20) if total_frames > 0 else 0
56              next_progress = progress_step
57              logger.info(f"[motion] start: {total_frames} frames @ {fps:.1f}fps "
58                          f"(~{duration/60:.1f} min), analyzing every {frame_interval}th
```

```
frame")
            while True:
                # Skip decoding for intermediate frames using cap.grab() for high
performance
                for _ in range(frame_interval - 1):
                    if not cap.grab():
                        break
                    frame_idx += 1

                ret, frame = cap.read()
                if not ret:
                    break

                # Process frame
                gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
                # Apply Gaussian Blur to smooth noise
                gray = cv2.GaussianBlur(gray, (21, 21), 0)

                t = frame_idx / fps
                timestamps.append(t)

                if prev_gray is not None:
                    # Frame absolute difference (highly optimized motion tracking)
                    diff = cv2.absdiff(prev_gray, gray)
                    _, thresh = cv2.threshold(diff, 25, 255, cv2.THRESH_BINARY)

                    # Dilate to fill gaps
                    thresh = cv2.dilate(thresh, None, iterations=2)

                    # Motion ratio
                    motion_ratio = cv2.countNonZero(thresh) / (thresh.shape[0] *
thresh.shape[1])
                    motion_signals.append(motion_ratio)
                else:
                    motion_signals.append(0.0)

                prev_gray = gray
                frame_idx += 1
                processed_count += 1

                if progress_step and frame_idx >= next_progress:
                    pct = 100.0 * frame_idx / total_frames
                    elapsed = time.time() - t_start
                    eta = elapsed / (frame_idx / total_frames) - elapsed if frame_idx else 0
                    logger.info(f"[motion] {pct:5.1f}% ? {frame_idx}/{total_frames} frames,
"
                                f"{processed_count} analyzed, {elapsed:.0f}s elapsed,
~{eta:.0f}s left")
                    next_progress += progress_step

                if max_frames is not None and processed_count >= max_frames:
                    logger.debug(f"Reached max_frames limit of {max_frames}. Halting indexer
frame parsing.")
                    break

                if frame_idx >= total_frames:
                    break

            cap.release()

            # Smooth signal using a moving average
            window_size = 5
            smoothed = []
            for i in range(len(motion_signals)):
                start = max(0, i - window_size // 2)
```

```
                end = min(len(motion_signals), i + window_size // 2 + 1)
                smoothed.append(sum(motion_signals[start:end]) / (end - start))

            # Detect active play boundaries
            in_segment = False
            segment_start = None
            raw_segments: List[Tuple[float, float]] = []

            for i, t in enumerate(timestamps):
                signal = smoothed[i]
                if signal > motion_threshold:
                    if not in_segment:
                        in_segment = True
                        segment_start = t
                else:
                    if in_segment:
                        in_segment = False
                        raw_segments.append((segment_start, t))

            # Handle video ending while in active segment
            if in_segment:
                raw_segments.append((segment_start, timestamps[-1]))

            # Post-Processing: 1. Merge segments separated by less than dead_time_merge_gap
            merged_segments: List[Tuple[float, float]] = []
            if raw_segments:
                curr_start, curr_end = raw_segments[0]
                for next_start, next_end in raw_segments[1:]:
                    if next_start - curr_end < dead_time_merge_gap:
                        # Merge
                        curr_end = next_end
                    else:
                        merged_segments.append((curr_start, curr_end))
                        curr_start, curr_end = next_start, next_end
                merged_segments.append((curr_start, curr_end))

            # Post-Processing: 2. Filter segments shorter than min_segment_duration
            final_segments: List[Tuple[float, float]] = [
                r for r in merged_segments if (r[1] - r[0]) >= min_segment_duration
            ]

            # Populate SQLite DB with segments and metadata events
            segments_data = []
            for start, end in final_segments:
                # Assign Server and Receiver randomly from pool
                server, receiver = random.sample(player_pool, 2)

                # Formulate dynamic rich tags
                dur = end - start
                metadata = {
                    "shot_count": random.randint(4, 25),
                    "intensity": "high" if dur > 12 else "medium",
                    "avg_speed_kmh": round(random.uniform(40, 85), 1)
                }

                segment_id = self.db.add_play_segment(video_id, start, end, server,
receiver, metadata)

                if segment_id:
                    # Add sub-segment events
                    # A: Serve Event (always at start)
                    serve_foul = random.random() < 0.15 # 15% chance of serve foul
                    self.db.add_event(
                        segment_id=segment_id,
                        timestamp=start + 0.5,
```

```
                        event_type="serve",
                        player=server,
                        details={"status": "foul" if serve_foul else "valid", "type":
"low_serve" if random.random() > 0.5 else "high_serve"}
                    )

                    # B: Smash Events (1 to 3 random smashes per segment)
                    smashes_count = random.randint(0, 3)
                    for _ in range(smashes_count):
                        smash_time = random.uniform(start + 1.5, end - 1.0)
                        smasher = random.choice([server, receiver])
                        self.db.add_event(
                            segment_id=segment_id,
                            timestamp=round(smash_time, 2),
                            event_type="smash",
                            player=smasher,
                            details={"speed_kmh": round(random.uniform(120, 250), 1)}
                        )

                    # C: Segment Termination Event (foul/winner)
                    foul_types = ["out_of_bounds", "net_touch", "double_hit", "fault"]
                    is_foul = random.random() > 0.4
                    error_player = random.choice([server, receiver])

                    self.db.add_event(
                        segment_id=segment_id,
                        timestamp=end - 0.2,
                        event_type="foul" if is_foul else "winner",
                        player=error_player,
                        details={"type": random.choice(foul_types)} if is_foul else
{"shot_type": "drop" if random.random() > 0.5 else "smash"}
                    )

                    # Store segment representation to return to caller
                    segments_data.append({
                        "id": segment_id,
                        "start_time": start,
                        "end_time": end,
                        "duration": dur,
                        "server": server,
                        "receiver": receiver,
                        "metadata": metadata
                    })

            return Success(value=segments_data)

    # Register the segmenter
    segmenter_registry.register(MotionPlaySegmenter)

```

11. assistant (2026-06-18T07:48:04.012Z)

I notice the `motion.py` `process_video` has a potential issue: `smashes_count = random.randint(0, 3)` then `random.uniform(start + 1.5, end - 1.0)` ? if the segment is short, `start+1.5 > end-1.0`, but `random.uniform` handles reversed bounds fine. Let me check the bigger hybrid segmenters now.

12. user (2026-06-18T07:48:04.657Z)

```
      """WASB-based hybrid play segmenter.

      Phase 2 uses **WASB** (nttcom/WASB-SBDT, MIT) shuttle-trajectory detection via
      WSL2 (backend/pipeline/detectors/wasb_runner.py). WASB ships pretrained
      badminton weights and runs today, so this is the live shuttle-motion substrate.

```

```
7       For the per-video *calibration* goal we mainly use Phase 2 (trajectory ->
8       candidate windows); the windowing thresholds (velocity_thresh / merge_gap /
9       min_window_duration) are what get tuned against golden rally labels.
10
11      The pipeline engine (healthcheck -> trajectory -> windows -> AI handoff -> DB)
12      lives in trajectory_hybrid.TrajectoryHybridSegmenter; this class only wires the
13      WASB runner and identity.
14      """
15      from typing import Any, Dict, Tuple
16
17      from backend.pipeline.segmenters.base import segmenter_registry
18      from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
19      from backend.pipeline.detectors.base import DetectorRunner
20      from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
21
22
23      class WasbHybridSegmenter(TrajectoryHybridSegmenter):
24          family = "wasb"
25          display_label = "WASB"
26
27          @property
28          def name(self) -> str:
29              return "wasb_hybrid"
30
31          @property
32          def description(self) -> str:
33              return ("Two-stage pipeline: WASB shuttle-trajectory detection (WSL2, pretrained
badminton "
34                      "weights) for precise candidate rallies + AI provider for semantic
boundaries.")
35
36          def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
37              cfg = WasbConfig.from_indexing_cfg(idx_cfg)
38              return WasbRunner(cfg), {"weights_path": cfg.weights_path}
39
40
41      segmenter_registry.register(WasbHybridSegmenter)
42
```

13. assistant (2026-06-18T07:48:07.635Z)

This is the core engine. Let me read `trajectory_hybrid.py`.

14. user (2026-06-18T07:48:07.999Z)

```
1       """Shared engine for trajectory-substrate hybrid segmenters.
2
3       tracknet_hybrid and wasb_hybrid were copy-paste twins ? CODE_MAP carried a
4       standing footgun: "any P3/P4 fix must be applied to BOTH". This base class is
5       now the single home for the whole pipeline shape:
6
7           healthcheck gate -> P2 trajectory -> candidate windows (+ persistence)
8           -> P3 AI-provider handoff over padded clips -> P4 DB population/linking
9
10      A concrete subclass supplies only its identity and runner wiring:
11      - ``family``        ? config section under ``indexing.*``, output subdir, and
12                            the ``<family>-hybrid-<model>`` detector tag in metadata.
13      - ``display_label`` ? human label for log lines ("WASB", "TrackNet").
14      - ``name`` / ``description`` properties (the registry contract).
15      - ``_build_runner(idx_cfg)`` ? returns (DetectorRunner, detector-specific
16                            detection_params extras such as weights_path).
17
18      This base is deliberately NOT registered; only concrete subclasses register.
19      """
```

```
    import os
    import time
    import logging
    import concurrent.futures
    from typing import Any, Dict, List, Optional, Tuple

    import cv2

    from backend.pipeline.segmenters.base import BasePlaySegmenter
    from backend.utils.video import get_video_duration, extract_video_chunk
    from backend.sports import sport_registry
    from backend.providers import provider_registry
    from backend.pipeline.detectors.base import DetectorRunner

    logger = logging.getLogger(__name__)


    def _fmt_ts(seconds: float) -> str:
        seconds = max(0, int(seconds))
        h, rem = divmod(seconds, 3600)
        m, s = divmod(rem, 60)
        return f"{h:02d}:{m:02d}:{s:02d}"


    class TrajectoryHybridSegmenter(BasePlaySegmenter):
        """Trajectory-detector hybrid: precise candidate rallies + AI semantic
boundaries."""

        #: config section under `indexing` (velocity_threshold lives there), the
        #: output subdir for trajectories, and the metadata detector-tag prefix.
        family: str = ""
        #: human-readable label used in log lines.
        display_label: str = ""

        def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
            """Return (runner, detector-specific detection_params extras)."""
            raise NotImplementedError

        @staticmethod
        def _probe_fps_width(video_path: str) -> tuple:
            """Return (fps, frame_width) for a video, with sensible fallbacks."""
            cap = cv2.VideoCapture(video_path)
            fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
            width = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
            cap.release()
            return (fps if fps > 0 else 30.0, width if width > 0 else 1280.0)

        @staticmethod
        def _shot_count_from(segment: Dict[str, Any], duration: float) -> int:
            """Provider-reported exchange count, else a duration-based estimate.

            One canonical implementation ? the twins had drifted (wasb relied on
            ``int(None)`` raising into the fallback; same result, by accident).
            """
            shot_count = segment.get("shuttle_exchange_count")
            if shot_count is None:
                return max(3, int(duration / 0.8))
            try:
                return int(shot_count)
            except (ValueError, TypeError):
                return max(3, int(duration / 0.8))

        # --- Fusion seams (identity by default ? wasb_hybrid/tracknet_hybrid unchanged)
-------
```

```
        def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
                                    frame_width: float, video_path: str,
                                    idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
            """Stats dict persisted into ``candidate_segments.stats`` for one window. The
BASE
            returns exactly the 4 motion keys, so the trajectory twins persist byte-
identical
            rows; ``FusionSegmenter`` overrides this to add ``net_crossings`` (+ person-cue
            features). ``points`` are the whole-video trajectory points
(TrajectoryPoint)."""
            return {
                "peak_velocity": cw["peak_velocity"],
                "mean_velocity": cw["mean_velocity"],
                "active_frame_count": cw["active_frame_count"],
                "track_id_count": cw["track_id_count"],
            }

        def _select_for_handoff(self, windows: List[Dict[str, Any]],
                                window_stats: List[Dict[str, Any]],
                                idx_cfg: Dict[str, Any]) -> List[int]:
            """Indices of the candidate windows that proceed to the AI handoff (and thus may
            become play_segments). The BASE sends ALL windows (no gating ? twins unchanged);
            ``FusionSegmenter`` overrides this to apply Gate A/B when thresholds are raised.
            Persisted candidates are NOT affected ? they remain the full superset for
offline
            re-scoring."""
            return list(range(len(windows)))

        def process_video(self, video_id: str, video_path: str,  # type: ignore[override]
                          player_pool: Optional[List[str]] = None,
                          max_frames: Optional[int] = None) -> List[Dict[str, Any]]:
            # override-ignore: the ABC declares the Result contract (Success|Failure)
            # but every live segmenter still returns a bare list ? the documented
            # deliberate-incremental gap (CODE_MAP gotchas; main.py handles both).
            label = self.display_label
            # --- Sport profile + AI provider wiring (identical across segmenters) ---
            sport_name = self.config.get("sport", "badminton")
            try:
                sport_profile = sport_registry.get(sport_name)
            except KeyError as e:
                logger.error(str(e))
                return []

            idx_cfg = self.config.get("indexing", {})
            # Fully-local, NO-AI mode (default OFF): the trajectory candidate windows become
the
            # rallies directly ? Phase 3's AI handoff is skipped, so no provider / API key
is
            # needed and nothing is uploaded. Used to run the local detector standalone
(e.g. to
            # measure it against the Gemini path, or to avoid the cloud entirely). With
            # FusionSegmenter the windows are still Gate-A/B-filtered via
`_select_for_handoff`.
            skip_ai = bool(idx_cfg.get("skip_ai_handoff", False))
            provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
            if skip_ai:
                model_name = "local-noai"
                logger.info("%s in FULLY-LOCAL mode (skip_ai_handoff=True): candidate
windows will "
                            "be emitted as rallies; no %s handoff, no API key needed.",
                            label, provider_name)
            else:
                model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
"gemini-2.5-flash"))
```

```
                api_key_env_var = f"{provider_name.upper()}_API_KEY"
                api_key = os.environ.get(api_key_env_var)
                if not api_key:
                    logger.error(
                        f"{api_key_env_var} environment variable is not set! "
                        f"To run the {provider_name} handoff, set your API key. "
                        f"In PowerShell: $env:{api_key_env_var}=\"your_api_key_here\""
                    )
                    return []
                try:
                    provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
                except KeyError as e:
                    logger.error(str(e))
                    return []

            video_duration = get_video_duration(video_path)
            if video_duration <= 0:
                logger.error("Invalid video duration.")
                return []
            fps, frame_width = self._probe_fps_width(video_path)

            # --- Runner config + windowing thresholds ---
            runner, runner_params = self._build_runner(idx_cfg)

            velocity_thresh = idx_cfg.get(self.family, {}).get("velocity_threshold", 0.02)
            merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
            min_window_duration = idx_cfg.get("min_action_window_duration", 2.0)
            # Bounded-windowing over-merge fix (W1): 0 = OFF (unchanged). See
IndexingConfig.
            max_window_duration = idx_cfg.get("max_window_duration", 0.0)
            window_min_split_gap = idx_cfg.get("window_min_split_gap", 0.5)
            window_hard_cap = idx_cfg.get("window_hard_cap", False)
            window_inactivity_end = idx_cfg.get("window_inactivity_end", 0.0)
            inactivity_rally_thresh = idx_cfg.get("inactivity_rally_thresh", 0.08)
            inactivity_min_density = idx_cfg.get("inactivity_min_density", 0.34)
            window_blob_fallback = idx_cfg.get("window_blob_fallback", False)
            window_blob_factor = idx_cfg.get("window_blob_factor", 4.0)
            handoff_padding = idx_cfg.get("ai_handoff_padding", 2.0)
            ai_max_workers = idx_cfg.get("ai_max_workers", 5)
            log_candidates = idx_cfg.get("log_yolo_candidates", True)
            store_candidates = idx_cfg.get("store_yolo_candidates", True)

            detection_params = {
                "detector": self.name,
                "velocity_thresh": velocity_thresh,
                "merge_gap": merge_gap,
                "min_action_window_duration": min_window_duration,
                **runner_params,
            }

            # --- Phase 1: env healthcheck (fail fast with a clear message) ---
            ok, msg = runner.healthcheck()
            if not ok:
                logger.error("%s WSL environment not ready: %s", label, msg)
                return []
            logger.info("%s WSL env OK: %s", label, msg)

            # --- Phase 2: trajectory -> candidate rally windows ---
            # Trajectory detectors process the whole video in one pass (they carry
            # their own frame buffering), so unlike yolo_hybrid we don't pre-chunk.
            t_start = time.time()
            out_dir = os.path.join("output", self.family)
            csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
            if not csv_path:
```

```
                logger.error("%s produced no trajectory; aborting.", label)
                return []

            points = runner.parse_trajectory_csv(csv_path)
            logger.info("Parsed %d trajectory points (%d visible).",
                        len(points), sum(1 for p in points if p.visible))

            all_candidate_windows = runner.trajectory_to_action_windows(
                points, fps=fps, frame_width=frame_width, chunk_start=0.0,
                velocity_thresh=velocity_thresh, merge_gap=merge_gap,
                min_window_duration=min_window_duration, chunk_index=0,
                max_window_duration=max_window_duration, min_split_gap=window_min_split_gap,
                window_hard_cap=window_hard_cap,
                window_inactivity_end=window_inactivity_end,
                inactivity_rally_thresh=inactivity_rally_thresh,
                inactivity_min_density=inactivity_min_density,
                window_blob_fallback=window_blob_fallback,
                window_blob_factor=window_blob_factor,
            )
            logger.info("Phase 2 (%s) completed in %.1fs. Candidate windows: %d",
                        label, time.time() - t_start, len(all_candidate_windows))

            if log_candidates:
                for cw in all_candidate_windows:
                    logger.info(f"[{label} rally]
{_fmt_ts(cw['start'])}-{_fmt_ts(cw['end'])} "
                                f"({cw['end'] - cw['start']:.1f}s) |
frames={cw['active_frame_count']} "
                                f"peak_v={cw['peak_velocity']:.3f}
mean_v={cw['mean_velocity']:.3f}")

            # Per-window stats via the enrichment hook ? base = the 4 motion keys; the
fusion
            # segmenter adds net_crossings + person-cue features. Computed once, reused for
both
            # persistence and the handoff gate below.
            window_stats = [
                self._enrich_candidate_stats(cw, points, fps, frame_width, video_path,
idx_cfg)
                for cw in all_candidate_windows
            ]

            # Persist raw candidates for the fine-tuning dataset (same table as YOLO).
            candidate_ids: List[Optional[int]] = [None] * len(all_candidate_windows)
            if store_candidates:
                for i, cw in enumerate(all_candidate_windows):
                    candidate_ids[i] = self.db.add_candidate_segment(
                        video_id, detector=self.name,
                        start_time=cw["start"], end_time=cw["end"],
                        chunk_index=cw["chunk_index"], confidence=min(1.0,
cw["peak_velocity"]),
                        stats=window_stats[i], detection_params=detection_params,
                    )

            if not all_candidate_windows:
                return []

            # --- Phase 3: AI provider handoff over padded candidate clips ---
            # Which windows reach the handoff (base = all; fusion may apply Gate A/B).
Persisted
            # candidates above stay the full superset ? only the served play_segments are
gated.
            handoff_indices = self._select_for_handoff(all_candidate_windows, window_stats,
idx_cfg)

```

```
            ai_segments: List[dict] = []
            if skip_ai:
                # Fully-local: the selected candidate windows ARE the rallies ? emit them
verbatim
                # (no clip extraction, no upload). Boundaries are the local detector's; shot
count
                # falls back to the duration-based estimate (no AI exchange count).
                ai_segments = [
                    {
                        "start_time": all_candidate_windows[wi]["start"],
                        "end_time": all_candidate_windows[wi]["end"],
                        "shuttle_exchange_count": None,
                        "shot_count_confidence": "LocalNoAI",
                        "_candidate_id": candidate_ids[wi],
                    }
                    for wi in handoff_indices
                ]
                logger.info("Phase 3 SKIPPED (fully-local): emitted %d candidate windows as
rallies "
                            "(no AI handoff).", len(ai_segments))
            else:
                handoff_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
                os.makedirs(handoff_dir, exist_ok=True)
                logger.info("Phase 3: %s validation on %d candidate windows...",
                            provider_name, len(handoff_indices))

                def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
                    w_start = max(0, window["start"] - handoff_padding)
                    w_end = min(video_duration, window["end"] + handoff_padding)
                    w_dur = w_end - w_start
                    w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
                    if not extract_video_chunk(video_path, w_start, w_dur, w_path,
reencode=True):
                        return []
                    prompt = sport_profile.get_prompt(chunk_duration=w_dur)
                    segments, _ = provider.analyze_video(w_path, prompt,
chunk_duration=w_dur,
                                                         label=f"Handoff {wi+1}")
                    valid = []
                    for seg in segments:
                        try:
                            seg["start_time"] = float(seg["start_time"]) + w_start
                            seg["end_time"] = float(seg["end_time"]) + w_start
                            seg["_candidate_id"] = candidate_ids[wi]
                            valid.append(seg)
                        except (ValueError, TypeError, KeyError) as e:
                            logger.warning("Dropping segment with unparseable timestamps: %s
? %s", seg, e)
                    try:
                        os.remove(w_path)
                    except OSError:
                        pass
                    return valid

                with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as
executor:
                    futures = [executor.submit(process_candidate_window, wi,
all_candidate_windows[wi])
                               for wi in handoff_indices]
                    for future in futures:
                        ai_segments.extend(future.result())
                try:
                    os.rmdir(handoff_dir)
                except OSError:
                    pass
```

```
311                 logger.info("Phase 3 completed. AI confirmed %d semantic play segments.",
len(ai_segments))
312
313             # --- Phase 4: DB population + candidate linking ---
314             segments_data = []
315             for segment in ai_segments:
316                 try:
317                     start = float(segment["start_time"])
318                     end = float(segment["end_time"])
319                     dur = end - start
320                     metadata = {
321                         "shot_count": self._shot_count_from(segment, dur),
322                         "shot_count_confidence": segment.get("shot_count_confidence",
"Unknown"),
323                         "detector": f"{self.family}-hybrid-{model_name}",
324                     }
325                     segment_id = self.db.add_play_segment(video_id, start, end, None, None,
metadata)
326                     if segment_id:
327                         candidate_id = segment.get("_candidate_id")
328                         if candidate_id is not None:
329                             self.db.link_candidate_to_segment(candidate_id, segment_id)
330                         segments_data.append({
331                             "id": segment_id, "start_time": start, "end_time": end,
332                             "duration": dur, "metadata": metadata,
333                         })
334                 except Exception as e:
335                     logger.warning("Skipping invalid segment struct: %s", e)
336
337             return segments_data
338
```

15. assistant (2026-06-18T07:48:10.409Z)

Now let me read the fusion segmenter (overrides), yolo_hybrid (different chunking path), and the validators.

16. user (2026-06-18T07:48:11.186Z)

```
1       """Two-stage rally segmenter: torchvision person detection + ByteTrack tracking
2       (Stage 1, local) feeding an AI provider for semantic boundaries (Stage 2).
3
4       Stage-1 detection now lives in `backend/pipeline/detectors/person_detector.py`
5       (`PersonDetector`, a reusable rally-cue producer ? fusion P1) so the SAME person cue
6       can be consumed by the future fusion segmenter; this segmenter just orchestrates
7       chunking, the AI-provider handoff, and DB population around it. Behaviour is unchanged.
8
9       Stage 1 originally used the AGPL-licensed ultralytics YOLO. To keep the hosted
10      service free of copyleft obligations (see docs/MONETIZATION_AUDIT.md and ADR-005),
11      detection uses torchvision's permissively-licensed (BSD) detectors and the
12      association/tracking that YOLO.track() used to bundle in is provided by
13      supervision's MIT-licensed ByteTrack.
14
15      The registry key stays "yolo_hybrid" for back-compat with existing configs and the
16      DB `detector` tags, even though no YOLO is involved anymore.
17      """
18      import os
19      import time
20      import logging
21      import concurrent.futures
22      from typing import List, Dict, Any, Tuple
23
24      from backend.pipeline.segmenters.base import BasePlaySegmenter, segmenter_registry
25      from backend.pipeline.detectors.person_detector import PersonDetector, _DEFAULT_DETECTOR
26      from backend.utils.video import get_video_duration, extract_video_chunk
```

```
    from backend.sports import sport_registry
    from backend.providers import provider_registry

    logger = logging.getLogger(__name__)


    def _fmt_ts(seconds: float) -> str:
        """Format seconds as HH:MM:SS for cross-verifying a timestamp against the video."""
        seconds = max(0, int(seconds))
        h, rem = divmod(seconds, 3600)
        m, s = divmod(rem, 60)
        return f"{h:02d}:{m:02d}:{s:02d}"


    class HybridYoloSegmenter(BasePlaySegmenter):
        def __init__(self, db, config):
            super().__init__(db, config)
            # Stage-1 person cue (detection + tracking + motion windows) ? a reusable
producer
            # (fusion P1). This segmenter orchestrates chunking + AI handoff + DB around it.
            self.person = PersonDetector(config)

        @property
        def name(self) -> str:
            return "yolo_hybrid"

        @property
        def description(self) -> str:
            return ("Two-stage pipeline: torchvision person detection + ByteTrack for fast "
                    "candidate filtering, then an AI provider for high-precision semantic
boundaries.")

        def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None, max_frames: int = None) -> List[Dict[str, Any]]:
            self.person.lazy_load_model()

            # Get sport profile config
            sport_name = self.config.get("sport", "badminton")
            try:
                sport_profile = sport_registry.get(sport_name)
            except KeyError as e:
                logger.error(str(e))
                return []

            # Get AI provider and model config
            idx_cfg = self.config.get("indexing", {})
            provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
            model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
"gemini-2.5-flash"))

            # Get appropriate API key based on the provider
            api_key_env_var = f"{provider_name.upper()}_API_KEY"
            api_key = os.environ.get(api_key_env_var)
            if not api_key:
                logger.error(
                    f"{api_key_env_var} environment variable is not set! "
                    f"To run the {provider_name} segmenter, set your API key. "
                    f"In PowerShell: $env:{api_key_env_var}=\"your_api_key_here\""
                )
                return []

            try:
                provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
```

```
86              except KeyError as e:
87                  logger.error(str(e))
88                  return []
89
90              logger.info(f"Initializing Hybrid Segmenter (torchvision + ByteTrack) for video:
{video_path}")
91
92              video_duration = get_video_duration(video_path)
93              if video_duration <= 0:
94                  logger.error("Invalid video duration.")
95                  return []
96
97              chunk_duration = idx_cfg.get("chunk_duration", 300)
98              chunk_overlap = idx_cfg.get("chunk_overlap", 30)
99              velocity_thresh = idx_cfg.get("yolo_velocity_threshold", 0.15)  # Fraction of
frame width per second
100             merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
101             frame_skip = idx_cfg.get("yolo_frame_skip", 3)
102             tracker_config = idx_cfg.get("yolo_tracker", "bytetrack.yaml")
103             handoff_padding = idx_cfg.get("ai_handoff_padding",
idx_cfg.get("gemini_handoff_padding", 2.0))
104             ai_max_workers = idx_cfg.get("ai_max_workers", idx_cfg.get("gemini_max_workers",
5))
105             log_candidates = idx_cfg.get("log_yolo_candidates", True)
106             store_candidates = idx_cfg.get("store_yolo_candidates", True)
107
108             # Snapshot of the params that produced these detections ? stored alongside each
109             # candidate so a fine-tuning run can reproduce / correlate against them.
110             detection_params = {
111                 "velocity_thresh": velocity_thresh,
112                 "merge_gap": merge_gap,
113                 "frame_skip": frame_skip,
114                 "tracker": tracker_config,
115                 "detector_model": idx_cfg.get("detector_model", _DEFAULT_DETECTOR),
116                 "detector_score_threshold": idx_cfg.get("detector_score_threshold", 0.5),
117                 "min_action_window_duration": idx_cfg.get("min_action_window_duration",
2.0),
118             }
119
120             chunks_dir = os.path.join("output", "chunks_yolo")
121             os.makedirs(chunks_dir, exist_ok=True)
122
123             step = chunk_duration - chunk_overlap
124             chunk_starts = []
125             t = 0.0
126             while t < video_duration:
127                 chunk_starts.append(t)
128                 t += step
129
130             num_chunks = len(chunk_starts)
131             logger.info(f"Video duration: {video_duration:.1f}s. Splitting into {num_chunks}
chunks.")
132
133             all_candidate_windows = []
134
135             t_start = time.time()
136             prefetch_workers = idx_cfg.get("yolo_prefetch_workers", 2)
137
138             if num_chunks > 1 and prefetch_workers > 0:
139                 # --- PIPELINED PROCESSING ---
140                 # GPU inference must stay sequential (single GPU), but ffmpeg chunk
141                 # extraction is pure I/O. We pre-extract upcoming chunks in background
142                 # threads so extraction of chunk N+1 overlaps with inference on chunk N.
143                 logger.info(f"Running detection Phase 2 with pipelined extraction
({prefetch_workers} prefetch workers)...")
```

```
144
145                 extract_executor =
concurrent.futures.ThreadPoolExecutor(max_workers=prefetch_workers)
146
147                 def _extract_chunk(ci: int, cs: float) -> Tuple[int, float, str, bool]:
148                     chunk_end = min(cs + chunk_duration, video_duration)
149                     actual_dur = chunk_end - cs
150                     chunk_path = os.path.join(chunks_dir, f"chunk_{video_id}_{ci}.mp4")
151                     success = extract_video_chunk(video_path, cs, actual_dur, chunk_path)
152                     return (ci, cs, chunk_path, success)
153
154                 # Submit all extractions upfront ? they'll queue and run as workers free up
155                 extract_futures = [
156                     extract_executor.submit(_extract_chunk, ci, cs)
157                     for ci, cs in enumerate(chunk_starts)
158                 ]
159
160                 # Process results in order: wait for extraction, then run inference on GPU
161                 for future in extract_futures:
162                     ci, chunk_start, chunk_path, success = future.result()
163                     chunk_end = min(chunk_start + chunk_duration, video_duration)
164                     logger.info(f"Processing Chunk {ci+1}/{num_chunks} ({chunk_start:.1f}s -
{chunk_end:.1f}s)...")
165
166                     if not success:
167                         logger.warning(f"Chunk {ci+1} extraction failed. Skipping.")
168                         continue
169
170                     windows = self.person.extract_action_windows_from_chunk(
171                         chunk_path, chunk_start, velocity_thresh, merge_gap,
172                         frame_skip=frame_skip, chunk_index=ci
173                     )
174                     logger.info(f"Found {len(windows)} candidate action windows in chunk
{ci+1}.")
175                     all_candidate_windows.extend(windows)
176
177                     try:
178                         os.remove(chunk_path)
179                     except Exception:
180                         pass
181
182                 extract_executor.shutdown(wait=False)
183             else:
184                 # --- SEQUENTIAL PROCESSING (single chunk or prefetch disabled) ---
185                 for ci, chunk_start in enumerate(chunk_starts):
186                     chunk_end = min(chunk_start + chunk_duration, video_duration)
187                     actual_dur = chunk_end - chunk_start
188                     chunk_path = os.path.join(chunks_dir, f"chunk_{video_id}_{ci}.mp4")
189
190                     logger.info(f"Processing Chunk {ci+1}/{num_chunks} ({chunk_start:.1f}s -
{chunk_end:.1f}s)...")
191                     success = extract_video_chunk(video_path, chunk_start, actual_dur,
chunk_path)
192                     if not success:
193                         continue
194
195                     windows = self.person.extract_action_windows_from_chunk(
196                         chunk_path, chunk_start, velocity_thresh, merge_gap,
197                         frame_skip=frame_skip, chunk_index=ci
198                     )
199                     logger.info(f"Found {len(windows)} candidate action windows in chunk
{ci+1}.")
200                     all_candidate_windows.extend(windows)
201
202                     try:
```

```
                        os.remove(chunk_path)
                    except Exception:
                        pass

            # Deduplicate overlapping candidate windows across chunks (chunks overlap by
            # chunk_overlap, so the same rally can surface in two adjacent chunks).
            def _merge_stats(a: Dict[str, Any], b: Dict[str, Any]) -> Dict[str, Any]:
                fa, fb = a["active_frame_count"], b["active_frame_count"]
                total = fa + fb or 1
                return {
                    "start": a["start"],
                    "end": max(a["end"], b["end"]),
                    "active_frame_count": fa + fb,
                    "peak_velocity": max(a["peak_velocity"], b["peak_velocity"]),
                    # Frame-weighted mean so longer sub-windows dominate proportionally.
                    "mean_velocity": (a["mean_velocity"] * fa + b["mean_velocity"] * fb) /
total,
                    "track_id_count": max(a["track_id_count"], b["track_id_count"]),
                    "chunk_index": a["chunk_index"],
                }

            if all_candidate_windows:
                all_candidate_windows.sort(key=lambda w: w["start"])
                merged_windows = [all_candidate_windows[0]]
                for current in all_candidate_windows[1:]:
                    last = merged_windows[-1]
                    if current["start"] <= last["end"]:  # Overlap
                        merged_windows[-1] = _merge_stats(last, current)
                    else:
                        merged_windows.append(current)
                all_candidate_windows = merged_windows

            logger.info(f"Phase 2 tracking completed in {time.time() - t_start:.1f}s.")
            logger.info(f"Total candidate windows: {len(all_candidate_windows)}")

            # Per-rally console log (final, full-video timestamps) for video cross-
verification.
            if log_candidates:
                for w in all_candidate_windows:
                    logger.info(
                        f"[rally] {_fmt_ts(w['start'])}-{_fmt_ts(w['end'])} "
                        f"({w['end'] - w['start']:.1f}s) | frames={w['active_frame_count']}
"
                        f"peak_v={w['peak_velocity']:.3f} mean_v={w['mean_velocity']:.3f} "
                        f"tracks={w['track_id_count']}"
                    )

            # Persist raw candidates for the fine-tuning dataset. Map window index ->
candidate_id
            # so confirmed AI segments can be linked back in Phase 4.
            candidate_ids = [None] * len(all_candidate_windows)
            if store_candidates:
                for i, w in enumerate(all_candidate_windows):
                    stats = {
                        "peak_velocity": w["peak_velocity"],
                        "mean_velocity": w["mean_velocity"],
                        "active_frame_count": w["active_frame_count"],
                        "track_id_count": w["track_id_count"],
                    }
                    # Use peak velocity (capped at 1.0) as a coarse generic confidence
score.
                    confidence = min(1.0, w["peak_velocity"])
                    candidate_ids[i] = self.db.add_candidate_segment(
                        video_id, detector="yolo_hybrid",
                        start_time=w["start"], end_time=w["end"],
```

```
                        chunk_index=w["chunk_index"], confidence=confidence,
                        stats=stats, detection_params=detection_params,
                    )

            try:
                os.rmdir(chunks_dir)
            except Exception:
                pass

            # Phase 3: AI Provider Handoff
            gemini_segments = []
            gemini_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
            os.makedirs(gemini_dir, exist_ok=True)

            logger.info(f"Starting Phase 3: AI Provider ({provider_name}) Validation on
{len(all_candidate_windows)} candidate windows...")

            def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
                w_start = max(0, window["start"] - handoff_padding)
                w_end = min(video_duration, window["end"] + handoff_padding)
                w_dur = w_end - w_start

                w_path = os.path.join(gemini_dir, f"handoff_{video_id}_{wi}.mp4")
                # Re-encode short handoff clips to avoid keyframe-boundary drift
                success = extract_video_chunk(video_path, w_start, w_dur, w_path,
reencode=True)
                if not success:
                    return []

                prompt = sport_profile.get_prompt(chunk_duration=w_dur)
                segments, metrics = provider.analyze_video(w_path, prompt,
chunk_duration=w_dur, label=f"Handoff {wi+1}")

                # Coordinate mapping ? shift local timestamps back to full-video time
                valid_segments = []
                for segment in segments:
                    try:
                        segment["start_time"] = float(segment["start_time"]) + w_start
                        segment["end_time"] = float(segment["end_time"]) + w_start
                        # Stamp the source candidate so Phase 4 can link
prediction->confirmation.
                        segment["_candidate_id"] = candidate_ids[wi]
                        valid_segments.append(segment)
                    except (ValueError, TypeError, KeyError) as e:
                        logger.warning(f"Dropping segment with unparseable timestamps:
{segment} ? {e}")

                try:
                    os.remove(w_path)
                except OSError:
                    pass

                return valid_segments

            with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as
executor:
                futures = [executor.submit(process_candidate_window, wi, w) for wi, w in
enumerate(all_candidate_windows)]
                # Iterate in submission order to preserve temporal ordering
                for future in futures:
                    gemini_segments.extend(future.result())

            try:
                os.rmdir(gemini_dir)
            except OSError:
```

```
321                 pass
322
323             logger.info(f"Phase 3 completed. AI confirmed {len(gemini_segments)} semantic
play segments.")
324
325             # Phase 4: DB Population
326             segments_data = []
327             for segment in gemini_segments:
328                 try:
329                     start = float(segment["start_time"])
330                     end = float(segment["end_time"])
331                     dur = end - start
332
333                     shot_count = segment.get("shuttle_exchange_count")
334                     if shot_count is None:
335                         shot_count = max(3, int(dur / 0.8))
336                     else:
337                         try:
338                             shot_count = int(shot_count)
339                         except (ValueError, TypeError):
340                             shot_count = max(3, int(dur / 0.8))
341
342                     metadata = {
343                         "shot_count": shot_count,
344                         "shot_count_confidence": segment.get("shot_count_confidence",
"Unknown"),
345                         "detector": f"yolo-hybrid-{model_name}"
346                     }
347
348                     segment_id = self.db.add_play_segment(video_id, start, end, None, None,
metadata)
349                     if segment_id:
350                         # Link back to the candidate that produced this segment. A candidate
351                         # may yield multiple AI segments (one-to-many); confirmed_segment_id
keeps
352                         # the last one ? the link table records "this candidate was
confirmed".
353                         candidate_id = segment.get("_candidate_id")
354                         if candidate_id is not None:
355                             self.db.link_candidate_to_segment(candidate_id, segment_id)
356                         segments_data.append({
357                             "id": segment_id,
358                             "start_time": start,
359                             "end_time": end,
360                             "duration": dur,
361                             "metadata": metadata
362                         })
363                 except Exception as e:
364                     logger.warning(f"Skipping invalid segment struct: {e}")
365
366             return segments_data
367
368     # Register the segmenter
369     segmenter_registry.register(HybridYoloSegmenter)
370
```

17. assistant (2026-06-18T07:48:20.542Z)

Now the fusion segmenter (the overrides with gating logic) and validators.

18. user (2026-06-18T07:48:21.356Z)

```
1       """Multi-signal fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2).
2
3       Shuttle stays PRIMARY: this reuses an existing trajectory substrate (WASB by default,
```

```
4       TrackNet optional) to seed candidate windows exactly as `wasb_hybrid`/`tracknet_hybrid`
5       do ? it subclasses the same `TrajectoryHybridSegmenter` engine. It then ENRICHES each
6       shuttle-seeded window via two overridable seams the engine exposes:
7
8       - ``_enrich_candidate_stats`` ? always adds the **net-crossing count** (Gate A signal,
9         GPU-free, computed from the trajectory we already have); and, only when the person cue
10        is explicitly enabled (``indexing.fusion.cue_flags.person``), the person-cue features
11        (`fusion_features`), persisting them into ``candidate_segments.stats`` for the learned
12        fusion (P3). The person path lazily builds ONE `PersonDetector` and only then loads
the
13        torchvision/supervision model ? so the default (person-OFF) path never loads a
detector
14        model nor touches the GPU. (torch/cv2 themselves are imported by the registry
regardless,
15        via the pre-existing yolo_hybrid ? person_detector chain; the lazy part is the model.)
16      - ``_select_for_handoff`` ? optional served Gate A/B: when
``indexing.fusion.min_crossings``
17        (and, with the person cue, ``min_players``) are raised above 0, sub-threshold windows
are
18        dropped from the AI handoff (the held-shuttle / one-sided-feed false-positive fix).
19        **Persisted candidates remain the full superset** regardless, so the offline
experiment
20        harness can re-score any variant.
21
22      Defaults reproduce the substrate exactly: net_crossings/person features are persisted
but
23      nothing is gated (thresholds 0, person OFF), so `FusionSegmenter` with default config
seeds
24      and hands off identically to `wasb_hybrid`. Registered as ``fusion_hybrid``, NOT the
default
25      segmenter ? it is opt-in. Promotion to default happens only on measured LOVO lift
through the
26      experiment harness (EXPERIMENT_HARNESS.md), never silently.
27      """
28      from typing import Any, Dict, List, Optional, Tuple
29
30      from backend.pipeline.segmenters.base import segmenter_registry
31      from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
32      from backend.pipeline.detectors.base import DetectorRunner
33      from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
34      from backend.pipeline.detectors.tracknet_runner import TrackNetRunner, TrackNetConfig
35      from backend.pipeline.detectors import rally_gate
36      from backend.pipeline.detectors import fusion_features as ff
37
38
39      class FusionSegmenter(TrajectoryHybridSegmenter):
40          """Shuttle trajectory (primary) + net-crossing Gate A + optional person cue (Gate
B)."""
41
42          family = "fusion"
43          display_label = "Fusion"
44
45          def __init__(self, db: Any, config: Any):
46              super().__init__(db, config)
47              # Built lazily only if the person cue is enabled (no detector model loaded
otherwise).
48              self._person: Optional[Any] = None
49              # Cache the play-axis/net estimate per trajectory so rally_gate doesn't re-
estimate it
50              # over the whole trajectory once per candidate window (it depends only on
`points`).
51              self._axis_cache_key: Optional[int] = None
52              self._axis_cache: Optional[tuple] = None
53
54          @property
```

```
55          def name(self) -> str:
56              return "fusion_hybrid"
57
58          @property
59          def description(self) -> str:
60              return ("Multi-signal fusion: shuttle trajectory (WASB/TrackNet) seeds candidate
rallies, "
61                      "net-crossing Gate A + optional person-occupancy Gate B adjudicate them,
and the "
62                      "AI provider sets semantic boundaries.")
63
64          def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
65              substrate = idx_cfg.get("fusion", {}).get("substrate", "wasb")
66              if substrate == "tracknet":
67                  cfg = TrackNetConfig.from_indexing_cfg(idx_cfg)
68                  return TrackNetRunner(cfg), {
69                      "weights_path": cfg.weights_path,
70                      "queue_length": cfg.queue_length,
71                      "fusion_substrate": "tracknet",
72                  }
73              wcfg = WasbConfig.from_indexing_cfg(idx_cfg)
74              return WasbRunner(wcfg), {"weights_path": wcfg.weights_path, "fusion_substrate":
"wasb"}
75
76          def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
77                                      frame_width: float, video_path: str,
78                                      idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
79              stats = super()._enrich_candidate_stats(cw, points, fps, frame_width,
video_path, idx_cfg)
80              # Gate-A signal ? net-crossing count for this window. Pure, GPU-free, always
persisted
81              # (single service feed crosses ~once; a real rally crosses >=2x). axis/net auto-
estimated
82              # over the whole trajectory by rally_gate (camera-agnostic); reused across
windows.
83              gated = rally_gate.gate_windows(points, [(cw["start"], cw["end"])], fps,
84                                              min_crossings=0,
estimate=self._axis_estimate(points))
85              stats["net_crossings"] = float(gated[0].crossings) if gated else 0.0
86
87              fcfg = idx_cfg.get("fusion", {})
88              if fcfg.get("cue_flags", {}).get("person", False):
89                  stats.update(self._person_features(cw, points, fps, frame_width, video_path,
fcfg))
90              return stats
91
92          # P3 (learned fusion) first-step landing note (per MULTI_SIGNAL_FUSION_PLAN +
NEXT_STEPS 2026-06-14):
93          # Person features (from _person_features when cue_flags.person) are now persisted
for the harness
94          # `compare` LOVO litmus on the 6 golden videos. Eager-person-compute optimisation
(compute only
95          # for shuttle-seeded windows + padding, not whole video) is already in place via the
lazy
96          # self._person + windowed detections_per_frame. Next: actual harness run + lift
measurement
97          # (owner/hardware) before any served promotion. Flag-gated (default person OFF). See
98          # docs/NEXT_STEPS.md "Rally-detection quality track" and EXPERIMENT_HARNESS.md for
discipline.
99
100         def _axis_estimate(self, points: List[Any]) -> tuple:
101             """Cache rally_gate's (axis, net, span) play-axis estimate per trajectory so
it's
102             computed once per video, not once per candidate window (it depends only on
```

```
points)."""
103             key = id(points)
104             if self._axis_cache_key != key or self._axis_cache is None:
105                 self._axis_cache_key = key
106                 self._axis_cache = rally_gate.estimate_play_axis(points)
107             est = self._axis_cache
108             assert est is not None
109             return est
110
111         def _person_features(self, cw: Dict[str, Any], points: List[Any], fps: float,
112                              frame_width: float, video_path: str,
113                              fcfg: Dict[str, Any]) -> Dict[str, float]:
114             """GPU path (person cue ON): run the person detector over this window's
ORIGINAL-video
115             frame range and compute the scale/translation-invariant person features. Lazily
builds
116             ONE PersonDetector. ? real-video verification is owner-side; this wiring is
mock-tested."""
117             # Lazy import so the default (person-OFF) path never pulls torch/cv2 through
this module.
118             from backend.pipeline.detectors.person_detector import PersonDetector
119             if self._person is None:
120                 self._person = PersonDetector(self.config)
121
122             frame_skip = self.config.get("indexing", {}).get("yolo_frame_skip", 3)
123             # round (not truncate): int() would map e.g. 1.999s*30 to frame 59 not 60 ? a
124             # window-edge off-by-one that drops the boundary frame from the shuttle
alignment.
125             start_f, end_f = round(cw["start"] * fps), round(cw["end"] * fps)
126             det = self._person.detections_per_frame(video_path, start_f, end_f, frame_skip)
127             fw = det["frame_width"] or frame_width
128             fh = det["frame_height"]
129             # If we can't trust the frame dims, every normalised feature would silently
collapse
130             # to ~0 ? return an explicit all-zero person-feature row instead of a half-baked
one.
131             if fw <= 0 or fh <= 0:
132                 return {k: 0.0 for k in ff.PERSON_FEATURE_NAMES}
133
134             poly = fcfg.get("court_polygon")
135             polygon = [(float(p[0]), float(p[1])) for p in poly] if poly else None
136             net = (fcfg.get("net_axis", "y"), float(fcfg.get("net_position", 0.5)))
137             shuttle_in = [p for p in points if start_f <= p.frame <= end_f]
138             return ff.compute_person_features(det["frames"], shuttle_in, fw, fh,
139                                               court_polygon=polygon, net=net)
140
141         def _select_for_handoff(self, windows: List[Dict[str, Any]],
142                                 window_stats: List[Dict[str, Any]],
143                                 idx_cfg: Dict[str, Any]) -> List[int]:
144             """Served Gate A/B over the (already-enriched) per-window stats. Default
thresholds
145             (0) keep every window ? identical to the substrate. Persisted candidates are
146             untouched (full superset).
147
148             NEVER-EMPTY safeguard: a video that had ?1 window must keep ?1 after gating. If
the
149             thresholds would drop EVERY window (e.g. a short, noisy or one-sided-feed
trajectory
150             where no segment clears ``min_crossings``), the gate falls back to the full
ungated
151             set rather than emit a video with zero rallies. This makes Gate A/B strictly
152             *pruning* ? it can shrink a candidate set but never zero it out ? so layering it
on
153             the substrate is "do no harm": the bounded-windowing / largest-internal-gap
split
```

```
154             below it always wins on an otherwise-empty video. Observed on real footage
155             (mahadevpura-1): a single mega-window split into pieces, none clearing the
crossings
156             gate ? without this guard the whole clip yielded 0 rallies (no reel)."""
157             fcfg = idx_cfg.get("fusion", {})
158             min_crossings = int(fcfg.get("min_crossings", 0))
159             min_players = int(fcfg.get("min_players", 0))
160             person_on = bool(fcfg.get("cue_flags", {}).get("person", False))
161
162             keep: List[int] = []
163             for i, st in enumerate(window_stats):
164                 if min_crossings > 0 and st.get("net_crossings", 0.0) < min_crossings:
165                     continue
166                 if person_on and min_players > 0 and st.get("players_in_court", 0.0) <
min_players:
167                     continue
168                 keep.append(i)
169
170             if not keep and window_stats:
171                 # Gating emptied a non-empty video ? fall back to the ungated set.
172                 return list(range(len(window_stats)))
173             return keep
174
175
176     segmenter_registry.register(FusionSegmenter)
177
```

19. assistant (2026-06-18T07:48:23.626Z)

Now the validators.

20. user (2026-06-18T07:48:24.586Z)

```
1       from abc import ABC, abstractmethod
2       from pydantic import BaseModel
3       from typing import List, Dict, Any
4
5       class ValidationResult(BaseModel):
6           validator_name: str
7           passed: bool
8           message: str
9           details: Dict[str, Any] = {}
10
11      class VideoValidator(ABC):
12          @property
13          @abstractmethod
14          def name(self) -> str:
15              """Unique identifier for this validator."""
16              pass
17
18          @property
19          @abstractmethod
20          def description(self) -> str:
21              """Human-readable explanation of what this validator checks."""
22              pass
23
24          @abstractmethod
25          def validate(self, video_path: str, config: Dict[str, Any], context: Dict[str, Any])
-> ValidationResult:
26              """
27              Executes validation.
28              - video_path: Local path to the MP4 file.
29              - config: Dict parsed from config.json.
30              - context: Dynamic context dict shared across validators (stores shared values
like ffprobe metadata).
```

```
31              """
32              pass
33
34      class ValidationRegistry:
35          def __init__(self):
36              self._validators: List[VideoValidator] = []
37
38          def register(self, validator: VideoValidator):
39              """Registers a new validator."""
40              self._validators.append(validator)
41
42          def run_all_with_context(self, video_path: str, config: Dict[str, Any]):
43              """Runs all validators and returns ``(results, context)``.
44
45              The per-run ``context`` accumulates shared values (notably
46              ``context['ffprobe_meta']`` produced by FormatValidator). ``run_all``
47              discards it; callers that want media metadata for an observability report
48              use this variant instead of re-probing.
49              """
50              results = []
51              context: Dict[str, Any] = {}  # Shared context (e.g. for caching ffprobe
outputs)
52              for val in self._validators:
53                  try:
54                      res = val.validate(video_path, config, context)
55                      results.append(res)
56                  except Exception as e:
57                      results.append(ValidationResult(
58                          validator_name=val.name,
59                          passed=False,
60                          message=f"Validator threw an unexpected error: {str(e)}"
61                      ))
62              return results, context
63
64          def run_all(self, video_path: str, config: Dict[str, Any]) ->
List[ValidationResult]:
65              """Runs all registered validators in sequence, accumulating results."""
66              results, _ = self.run_all_with_context(video_path, config)
67              return results
68
69      # Shared global registry
70      registry = ValidationRegistry()
71
```

21. user (2026-06-18T07:48:25.062Z)

```
1       import subprocess
2       from backend.pipeline.validators.base import VideoValidator, ValidationResult
3       from typing import Dict, Any
4
5       class PTSContinuityValidator(VideoValidator):
6           @property
7           def name(self) -> str:
8               return "pts_continuity_check"
9
10          @property
11          def description(self) -> str:
12              return "Analyzes Presentation Time Stamps (PTS) of keyframes to check for
timeline gaps, jumps, or editing splices."
13
14          def validate(self, video_path: str, config: Dict[str, Any], context: Dict[str, Any])
-> ValidationResult:
15              # Command to extract packet PTS and flags (extremely fast, works on H.264 &
H.265)
16              cmd = [
```

```
                "ffprobe",
                "-v", "error",
                "-select_streams", "v:0",
                "-show_entries", "packet=pts_time,flags",
                "-of", "csv=p=0",
                video_path
            ]

            try:
                result = subprocess.run(cmd, capture_output=True, text=True, check=True)
                output = result.stdout.strip()

                if not output:
                    return ValidationResult(
                        validator_name=self.name,
                        passed=False,
                        message="No packet timestamps could be extracted from the video
stream."
                    )

                timestamps = []
                for line in output.split("\n"):
                    line = line.strip()
                    if line:
                        parts = line.split(",")
                        if len(parts) == 2:
                            pts_str, flags = parts
                            if "K" in flags: # Check if keyframe
                                try:
                                    timestamps.append(float(pts_str))
                                except ValueError:
                                    continue

                if len(timestamps) < 2:
                    return ValidationResult(
                        validator_name=self.name,
                        passed=True,
                        message="Video is too short to perform a PTS continuity check.",
                        details={"keyframes_count": len(timestamps)}
                    )

                # Analyze gaps
                max_gap = 0.0
                backward_jumps = 0
                gaps = []

                for i in range(1, len(timestamps)):
                    gap = timestamps[i] - timestamps[i-1]
                    gaps.append(gap)
                    if gap < 0:
                        backward_jumps += 1
                    if gap > max_gap:
                        max_gap = gap

                # An continuous GoPro recording usually has keyframe intervals (GOP size) of
1.0 to 4.0 seconds.
                # If there's a keyframe gap larger than 8.0 seconds or a negative gap
(backward jump), it's highly likely spliced.
                details = {
                    "keyframes_count": len(timestamps),
                    "max_keyframe_gap_seconds": round(max_gap, 2),
                    "backward_jumps_detected": backward_jumps,
                    "first_keyframe": timestamps[0],
                    "last_keyframe": timestamps[-1]
                }
```

```
79
80                  if backward_jumps > 0:
81                      return ValidationResult(
82                          validator_name=self.name,
83                          passed=False,
84                          message="Video integrity compromise: Backward timeline jumps
(splicing) detected.",
85                          details=details
86                      )
87
88                  # Threshold for sudden gap (e.g. 10.0 seconds)
89                  if max_gap > 10.0:
90                      return ValidationResult(
91                          validator_name=self.name,
92                          passed=False,
93                          message=f"Video integrity compromise: Detected a timestamp gap of
{round(max_gap, 2)}s, indicating edited or missing content.",
94                          details=details
95                      )
96
97                  return ValidationResult(
98                      validator_name=self.name,
99                      passed=True,
100                     message="PTS timeline is continuous with no abnormal gaps or backward
jumps.",
101                     details=details
102                 )
103
104             except subprocess.CalledProcessError as e:
105                 return ValidationResult(
106                     validator_name=self.name,
107                     passed=False,
108                     message="Failed to execute ffprobe PTS analysis.",
109                     details={"error": e.stderr}
110                 )
111             except Exception as e:
112                 return ValidationResult(
113                     validator_name=self.name,
114                     passed=False,
115                     message=f"Unexpected error in PTS validation: {str(e)}"
116                 )
117
```

22. user (2026-06-18T07:48:25.638Z)

```
1       import cv2
2       from backend.pipeline.validators.base import VideoValidator, ValidationResult
3       from typing import Dict, Any
4
5       class SceneCutDensityValidator(VideoValidator):
6           @property
7           def name(self) -> str:
8               return "scene_cut_check"
9
10          @property
11          def description(self) -> str:
12              return "Analyzes visual frames to detect abrupt scene cuts, which indicate an
edited or compiled video."
13
14          def validate(self, video_path: str, config: Dict[str, Any], context: Dict[str, Any])
-> ValidationResult:
15              cap = cv2.VideoCapture(video_path)
16              if not cap.isOpened():
17                  return ValidationResult(
18                      validator_name=self.name,
```

```
                    passed=False,
                    message="Failed to open video file using OpenCV."
                )

            fps = cap.get(cv2.CAP_PROP_FPS)
            total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
            duration = total_frames / fps if fps > 0 else 0

            if duration <= 0:
                cap.release()
                return ValidationResult(
                    validator_name=self.name,
                    passed=False,
                    message="Invalid video duration."
                )

            # Sample at 2 frames per second to keep it fast
            sample_interval = max(1, int(fps / 2))

            cuts_detected = 0
            prev_hist = None

            frame_idx = 0
            sampled_count = 0

            # Adjust histogram comparison thresholds
            # Using correlation: 1.0 is perfect match, < 0.6 indicates a sudden massive
scene change
            cut_threshold = 0.6

            while True:
                cap.set(cv2.CAP_PROP_POS_FRAMES, frame_idx)
                ret, frame = cap.read()
                if not ret:
                    break

                # Calculate color histogram
                hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
                hist = cv2.calcHist([hsv], [0, 1], None, [50, 60], [0, 180, 0, 256])
                cv2.normalize(hist, hist, alpha=0, beta=1, norm_type=cv2.NORM_MINMAX)

                if prev_hist is not None:
                    # Compare correlation
                    score = cv2.compareHist(prev_hist, hist, cv2.HISTCMP_CORREL)
                    if score < cut_threshold:
                        cuts_detected += 1

                prev_hist = hist
                sampled_count += 1
                frame_idx += sample_interval

                # Cap check at 100 frames max (e.g. first 50 seconds or evenly spread) to
keep validation instant
                if frame_idx >= total_frames or sampled_count > 100:
                    break

            cap.release()

            # Compute cuts per minute
            sampled_duration = (sampled_count * sample_interval) / fps if fps > 0 else 1
            cuts_per_minute = (cuts_detected / sampled_duration) * 60 if sampled_duration >
0 else 0

            details = {
                "cuts_detected": cuts_detected,
```

```
81                  "sampled_frames": sampled_count,
82                  "cuts_per_minute": round(cuts_per_minute, 3)
83              }
84
85              val_cfg = config.get("validation", {})
86              max_cuts_rate = val_cfg.get("max_scene_cuts_per_minute", 0.5)
87
88              if cuts_per_minute > max_cuts_rate:
89                  return ValidationResult(
90                      validator_name=self.name,
91                      passed=False,
92                      message=f"Video splicing compromise: Detected {cuts_detected} cuts
({round(cuts_per_minute, 2)} cuts/min), which exceeds the continuous raw file limit of
{max_cuts_rate} cuts/min.",
93                      details=details
94                  )
95
96              return ValidationResult(
97                  validator_name=self.name,
98                  passed=True,
99                  message="Scene continuity check passed. No abnormal visual cuts detected.",
100                 details=details
101             )
102
```

23. assistant (2026-06-18T07:48:31.669Z)

Now the remaining validators.

24. user (2026-06-18T07:48:32.720Z)

```
1       import subprocess
2       import json
3       import os
4       from backend.pipeline.validators.base import VideoValidator, ValidationResult
5       from typing import Dict, Any
6
7       class FormatValidator(VideoValidator):
8           @property
9           def name(self) -> str:
10              return "format_check"
11
12          @property
13          def description(self) -> str:
14              return "Verifies the video is a valid, uncorrupted MP4 container and extracts
metadata."
15
16          def validate(self, video_path: str, config: Dict[str, Any], context: Dict[str, Any])
-> ValidationResult:
17              if not os.path.exists(video_path):
18                  return ValidationResult(
19                      validator_name=self.name,
20                      passed=False,
21                      message=f"File not found: {video_path}"
22                  )
23
24              cmd = [
25                  "ffprobe",
26                  "-v", "error",
27                  "-select_streams", "v:0",
28                  "-show_format",
29                  "-show_streams",
30                  "-of", "json",
31                  video_path
32              ]
```

```
33
34              try:
35                  result = subprocess.run(cmd, capture_output=True, text=True, check=True)
36                  info = json.loads(result.stdout)
37
38                  if not info.get("streams") or not info.get("format"):
39                      return ValidationResult(
40                          validator_name=self.name,
41                          passed=False,
42                          message="Failed to retrieve valid video streams from the file."
43                      )
44
45                  # Store the metadata in context for subsequent validators to utilize
46                  context["ffprobe_meta"] = info
47
48                  stream = info["streams"][0]
49                  fmt = info["format"]
50
51                  details = {
52                      "format_name": fmt.get("format_name"),
53                      "duration": float(fmt.get("duration", 0)),
54                      "size_bytes": int(fmt.get("size", 0)),
55                      "codec_name": stream.get("codec_name"),
56                  }
57
58                  # Check if container name includes mp4
59                  if "mp4" not in fmt.get("format_name", "").lower():
60                      return ValidationResult(
61                          validator_name=self.name,
62                          passed=False,
63                          message=f"Invalid format: Found '{fmt.get('format_name')}', but MP4
is required.",
64                          details=details
65                      )
66
67                  return ValidationResult(
68                      validator_name=self.name,
69                      passed=True,
70                      message="Video format is valid MP4 container.",
71                      details=details
72                  )
73
74              except subprocess.CalledProcessError as e:
75                  return ValidationResult(
76                      validator_name=self.name,
77                      passed=False,
78                      message="Video stream is corrupted or unreadable by ffprobe.",
79                      details={"error": e.stderr}
80                  )
81              except Exception as e:
82                  return ValidationResult(
83                      validator_name=self.name,
84                      passed=False,
85                      message=f"Unexpected error running ffprobe: {str(e)}"
86                  )
87
```

25. user (2026-06-18T07:48:33.208Z)

```
1       from backend.pipeline.validators.base import VideoValidator, ValidationResult
2       from typing import Dict, Any
3
4       class ResolutionFPSValidator(VideoValidator):
5           @property
6           def name(self) -> str:
```

```
            return "resolution_fps_check"

        @property
        def description(self) -> str:
            return "Checks if the video meets minimum resolution (e.g. 720p) and frame rate
(e.g. 30fps) requirements."

        def validate(self, video_path: str, config: Dict[str, Any], context: Dict[str, Any])
-> ValidationResult:
            meta = context.get("ffprobe_meta")
            if not meta or not meta.get("streams"):
                return ValidationResult(
                    validator_name=self.name,
                    passed=False,
                    message="Video metadata is missing. Run format_check validator first."
                )

            stream = meta["streams"][0]
            width = int(stream.get("width", 0))
            height = int(stream.get("height", 0))

            # Parse frame rate (e.g. "30/1" or "30000/1001")
            fps_str = stream.get("r_frame_rate", "0/1")
            try:
                num, den = map(int, fps_str.split("/"))
                fps = num / den if den != 0 else 0
            except ValueError:
                fps = 0.0

            details = {
                "width": width,
                "height": height,
                "fps": round(fps, 2)
            }

            # Check bounds from config
            val_cfg = config.get("validation", {})
            min_w = val_cfg.get("min_resolution_width", 1280)
            min_h = val_cfg.get("min_resolution_height", 720)
            min_fps = val_cfg.get("min_fps", 29.9)

            if width < min_w or height < min_h:
                return ValidationResult(
                    validator_name=self.name,
                    passed=False,
                    message=f"Low resolution: Found {width}x{height}, but minimum is
{min_w}x{min_h}.",
                    details=details
                )

            if fps < min_fps:
                return ValidationResult(
                    validator_name=self.name,
                    passed=False,
                    message=f"Low frame rate: Found {round(fps, 2)} FPS, but minimum is
{min_fps} FPS.",
                    details=details
                )

            return ValidationResult(
                validator_name=self.name,
                passed=True,
                message="Video resolution and frame rate meet minimum standards.",
                details=details
            )
```

68

26. user (2026-06-18T07:48:34.012Z)

```
1       import cv2
2       from backend.pipeline.validators.base import VideoValidator, ValidationResult
3       from typing import Dict, Any
4
5       class BlurValidator(VideoValidator):
6           @property
7           def name(self) -> str:
8               return "blur_check"
9
10          @property
11          def description(self) -> str:
12              return "Samples frames across the video and checks if the footage is excessively
blurry or out of focus."
13
14          def validate(self, video_path: str, config: Dict[str, Any], context: Dict[str, Any])
-> ValidationResult:
15              cap = cv2.VideoCapture(video_path)
16              if not cap.isOpened():
17                  return ValidationResult(
18                      validator_name=self.name,
19                      passed=False,
20                      message="Failed to open video file for blur analysis."
21                  )
22
23              total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
24              if total_frames <= 0:
25                  cap.release()
26                  return ValidationResult(
27                      validator_name=self.name,
28                      passed=False,
29                      message="Invalid frame count."
30                  )
31
32              # Sample 15 frames evenly distributed across the video
33              num_samples = 15
34              step = max(1, total_frames // num_samples)
35
36              variances = []
37
38              for i in range(num_samples):
39                  frame_idx = i * step
40                  cap.set(cv2.CAP_PROP_POS_FRAMES, frame_idx)
41                  ret, frame = cap.read()
42                  if not ret:
43                      continue
44
45                  gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
46                  # Calculate the Laplacian variance (standard focus metric)
47                  laplacian_var = cv2.Laplacian(gray, cv2.CV_64F).var()
48                  variances.append(laplacian_var)
49
50              cap.release()
51
52              if not variances:
53                  return ValidationResult(
54                      validator_name=self.name,
55                      passed=False,
56                      message="Could not read frames to perform blur verification."
57                  )
58
59              avg_variance = sum(variances) / len(variances)
```

```
60
61              details = {
62                  "avg_laplacian_variance": round(avg_variance, 2),
63                  "min_laplacian_variance": round(min(variances), 2),
64                  "max_laplacian_variance": round(max(variances), 2),
65                  "sampled_frames_count": len(variances)
66              }
67
68              val_cfg = config.get("validation", {})
69              min_blur_threshold = val_cfg.get("min_blur_threshold", 100.0)
70
71              if avg_variance < min_blur_threshold:
72                  return ValidationResult(
73                      validator_name=self.name,
74                      passed=False,
75                      message=f"Video quality rejected: Focus check failed. Average sharpness
score of {round(avg_variance, 1)} is below the minimum required threshold of
{min_blur_threshold}.",
76                      details=details
77                  )
78
79              return ValidationResult(
80                  validator_name=self.name,
81                  passed=True,
82                  message="Video sharpness meets focus requirements.",
83                  details=details
84              )
85
```

27. user (2026-06-18T07:48:34.321Z)

```
1       import cv2
2       import numpy as np
3       from backend.pipeline.validators.base import VideoValidator, ValidationResult
4       from typing import Dict, Any
5
6       class RelevanceValidator(VideoValidator):
7           @property
8           def name(self) -> str:
9               return "relevance_check"
10
11          @property
12          def description(self) -> str:
13              return "Checks if the video content is relevant to badminton by scanning for
court color palettes and geometric features."
14
15          def validate(self, video_path: str, config: Dict[str, Any], context: Dict[str, Any])
-> ValidationResult:
16              cap = cv2.VideoCapture(video_path)
17              if not cap.isOpened():
18                  return ValidationResult(
19                      validator_name=self.name,
20                      passed=False,
21                      message="Failed to open video file for relevance validation."
22                  )
23
24              total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
25              if total_frames <= 0:
26                  cap.release()
27                  return ValidationResult(
28                      validator_name=self.name,
29                      passed=False,
30                      message="Invalid frame count."
31                  )
32
```

```
            # Sample 10 frames across the video
            num_samples = 10
            step = max(1, total_frames // num_samples)

            green_ratios = []

            # Get sport profile config
            sport_name = config.get("sport", "badminton")
            from backend.sports import sport_registry
            try:
                sport_profile = sport_registry.get(sport_name)
                sport_rel_cfg = sport_profile.get_relevance_config()
            except KeyError:
                sport_rel_cfg = {}

            val_cfg = config.get("validation", {})
            rel_cfg = val_cfg.get("relevance", {})

            # Read bounds from sport profile with fallback to user config/defaults
            hsv_lower = np.array(rel_cfg.get("court_green_lower",
sport_rel_cfg.get("court_green_lower", [35, 40, 40])))
            hsv_upper = np.array(rel_cfg.get("court_green_upper",
sport_rel_cfg.get("court_green_upper", [85, 255, 255])))
            min_court_ratio = rel_cfg.get("court_pixels_min_ratio",
sport_rel_cfg.get("court_pixels_min_ratio", 0.05))

            for i in range(num_samples):
                frame_idx = i * step
                cap.set(cv2.CAP_PROP_POS_FRAMES, frame_idx)
                ret, frame = cap.read()
                if not ret:
                    continue

                # Downscale frame for speed
                h, w = frame.shape[:2]
                small_frame = cv2.resize(frame, (320, 240))

                # Convert to HSV color space
                hsv = cv2.cvtColor(small_frame, cv2.COLOR_BGR2HSV)

                # Mask court green pixels
                mask = cv2.inRange(hsv, hsv_lower, hsv_upper)
                green_pixels = cv2.countNonZero(mask)
                total_pixels = mask.shape[0] * mask.shape[1]

                ratio = green_pixels / total_pixels
                green_ratios.append(ratio)

            cap.release()

            if not green_ratios:
                return ValidationResult(
                    validator_name=self.name,
                    passed=False,
                    message="Failed to extract frames for relevance checking."
                )

            avg_green_ratio = sum(green_ratios) / len(green_ratios)

            details = {
                "avg_green_court_ratio": round(avg_green_ratio, 4),
                "max_green_court_ratio": round(max(green_ratios), 4),
                "min_green_court_ratio": round(min(green_ratios), 4),
                "green_threshold": min_court_ratio
            }
```

```
95
96              # If a certain amount of court color is visible, consider it relevant
97              if avg_green_ratio < min_court_ratio:
98                  return ValidationResult(
99                      validator_name=self.name,
100                     passed=False,
101                     message=f"Video rejected: Relevance check failed. The scene does not
contain a recognizable {sport_name} court (court area found: {round(avg_green_ratio * 100, 2)}%
vs min required {round(min_court_ratio * 100, 2)}%).",
102                     details=details
103                 )
104
105             return ValidationResult(
106                 validator_name=self.name,
107                 passed=True,
108                 message=f"Video identified as relevant to {sport_name} ({sport_name} court
structures found).",
109                 details=details
110             )
111
```

28. assistant (2026-06-18T07:48:42.277Z)

I've now read all the validators. Let me focus on a couple of suspect areas. First, let me check the gemini.py segmenter's `_parse_time` to compare, and look at the `tracknet_hybrid.py`. But more importantly, let me investigate two potential defects I've spotted:

1. In `compiler.py`, `_merge_overlapping` parses and validates `e > s`, but then `compile_highlights` re-parses with `_parse_time` again AND re-runs all the validation. The merge happens BEFORE the duration clamp. Let me think about the padding overlap problem more carefully.

2. The `process_candidate_window` in trajectory/yolo: `w_start = max(0, window["start"] - handoff_padding)` ? `max(0, ...)` with int 0 vs float. That's fine.

Let me look at the merge math issue more closely. The key concern: `_merge_overlapping` merges segments that overlap/abut, but the BEGIN/END padding is applied AFTER merging per-segment. So two segments that are within `begin_padding + end_padding` of each other won't be merged, and their padded clips will overlap ? replaying footage. Let me verify this is a real, documented contract concern by checking the tests.

29. user (2026-06-18T07:48:42.864Z)

tests\test_compiler.py

30. user (2026-06-18T07:48:43.760Z)

No files found

31. user (2026-06-18T07:48:44.195Z)

```
1       """TrackNet-based hybrid play segmenter.
2
3       Phase 2 (candidate generation) uses a TrackNetV4 *shuttle* trajectory computed
4       in WSL2 (backend/pipeline/detectors/tracknet_runner.py, ADR-001). Rationale: a
5       tiny, fast, motion-blurred shuttle is exactly what general detectors miss; a
6       purpose-built trajectory model localises it far more precisely, giving cleaner
7       candidate rallies than person-velocity heuristics.
8
9       The pipeline engine (healthcheck -> trajectory -> windows -> AI handoff -> DB)
10      lives in trajectory_hybrid.TrajectoryHybridSegmenter; this class only wires the
11      TrackNet runner and identity.
12      """
13      from typing import Any, Dict, Tuple
```

```
14
15      from backend.pipeline.segmenters.base import segmenter_registry
16      from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
17      from backend.pipeline.detectors.base import DetectorRunner
18      from backend.pipeline.detectors.tracknet_runner import TrackNetRunner, TrackNetConfig
19
20
21      class TrackNetHybridSegmenter(TrajectoryHybridSegmenter):
22          family = "tracknet"
23          display_label = "TrackNet"
24
25          @property
26          def name(self) -> str:
27              return "tracknet_hybrid"
28
29          @property
30          def description(self) -> str:
31              return ("Two-stage pipeline: TrackNetV4 shuttle-trajectory detection (WSL2) for
precise "
32                      "candidate rallies + AI provider for high-precision semantic
boundaries.")
33
34          def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
35              cfg = TrackNetConfig.from_indexing_cfg(idx_cfg)
36              return TrackNetRunner(cfg), {
37                  "weights_path": cfg.weights_path,
38                  "queue_length": cfg.queue_length,
39              }
40
41
42      segmenter_registry.register(TrackNetHybridSegmenter)
43
```

32. assistant (2026-06-18T07:48:46.606Z)

Let me read the compiler test to understand the contract that's being asserted, and check the gemini segmenter's `_parse_time` for the parallel logic mentioned.

33. user (2026-06-18T07:48:47.201Z)

```
1       """VideoCompiler coverage (previously zero ? the user-facing highlight-reel output path,
2       where the f-string SyntaxError lived undetected). ffmpeg/ffprobe are mocked."""
3       import os
4       import subprocess
5       from unittest.mock import patch, MagicMock
6
7       import pytest
8
9       from backend.pipeline.stitcher.compiler import VideoCompiler
10
11
12      def test_parse_time_formats():
13          f = VideoCompiler._parse_time
14          assert f(12.5) == 12.5
15          assert f("12.5") == 12.5
16          assert f("1:30") == 90.0
17          assert f("1:00:00") == 3600.0
18          assert f("garbage") == 0.0
19
20
21      def test_merge_overlapping_collapses_duplicates_and_overlaps():
22          m = VideoCompiler._merge_overlapping
23          # Exact duplicates + near-duplicates (the "every rally twice" re-run bug) collapse
to one each.
```

```
        assert m([(28.2, 34.5), (28.5, 34.5), (40.5, 45.5), (40.5, 45.5)]) == [(28.2, 34.5),
(40.5, 45.5)]
        # Unsorted input is sorted; an overlapping range extends the previous.
        assert m([(100.0, 110.0), (0.0, 5.0), (104.0, 120.0)]) == [(0.0, 5.0), (100.0,
120.0)]
        # Genuinely distinct (gapped) rallies are preserved, not merged.
        assert m([(0.0, 5.0), (6.0, 9.0)]) == [(0.0, 5.0), (6.0, 9.0)]
        # Degenerate rows (end <= start) are dropped; string timestamps are accepted.
        assert m([("0:10", "0:20"), (30.0, 30.0), (31.0, 29.0)]) == [(10.0, 20.0)]


    def test_compile_dedups_before_stitching(tmp_path):
        """A segment list with every rally twice must yield ONE clip per unique rally."""
        comp = VideoCompiler(str(tmp_path / "out"))
        raw = tmp_path / "raw.mp4"
        raw.write_bytes(b"x")
        trims = []

        def fake_run(cmd, *a, **k):
            if cmd[0] == "ffprobe":
                return MagicMock(stdout="100.0\n", returncode=0)
            out = cmd[-1]
            if isinstance(out, str) and out.endswith(".mp4"):
                open(out, "wb").write(b"clip")
                # count per-clip trims (the concat list file input is not an .mp4 output)
                if "-i" in cmd and cmd[cmd.index("-i") + 1] == str(raw):
                    trims.append(cmd)
            return MagicMock(returncode=0, stdout=b"", stderr=b"")

        dup_segments = [(0.0, 5.0), (0.2, 5.0), (10.0, 15.0), (10.0, 15.0), (20.0, 25.0)]
        with patch.object(subprocess, "run", side_effect=fake_run):
            comp.compile_highlights(str(raw), dup_segments, "reel.mp4")
        # 5 input segments collapse to 3 unique rallies -> 3 per-clip trims, not 5.
        assert len(trims) == 3, f"expected 3 deduped trims, got {len(trims)}"


    def test_compile_basenames_output_and_stitches(tmp_path):
        comp = VideoCompiler(str(tmp_path / "out"))
        raw = tmp_path / "raw.mp4"
        raw.write_bytes(b"x")

        calls = []

        def fake_run(cmd, *a, **k):
            calls.append(cmd)
            # Create whatever output file ffmpeg/ffprobe was told to write so existence
checks pass.
            if cmd[0] == "ffprobe":
                return MagicMock(stdout="100.0\n", returncode=0)
            out = cmd[-1]
            if isinstance(out, str) and out.endswith((".mp4",)):
                open(out, "wb").write(b"clip")
            return MagicMock(returncode=0, stdout=b"", stderr=b"")

        with patch.object(subprocess, "run", side_effect=fake_run):
            # Traversal in the name must be stripped (defense-in-depth basename).
            out_path = comp.compile_highlights(str(raw), [(0.0, 5.0), (10.0, 15.0)],
"../reel.mp4")

        assert os.path.basename(out_path) == "reel.mp4"
        assert os.path.dirname(out_path).endswith("out")
        # Per-clip trims + one concat = at least 3 ffmpeg invocations (plus an ffprobe).
        assert sum(1 for c in calls if c[0] == "ffmpeg") >= 3


```

```
85      def test_compile_raises_on_no_segments(tmp_path):
86          comp = VideoCompiler(str(tmp_path / "out"))
87          raw = tmp_path / "raw.mp4"
88          raw.write_bytes(b"x")
89          with pytest.raises(ValueError):
90              comp.compile_highlights(str(raw), [], "reel.mp4")
91
92
93      def test_compile_raises_when_source_missing(tmp_path):
94          comp = VideoCompiler(str(tmp_path / "out"))
95          with pytest.raises(FileNotFoundError):
96              comp.compile_highlights(str(tmp_path / "nope.mp4"), [(0.0, 5.0)], "reel.mp4")
97
```

34. assistant (2026-06-18T07:48:58.742Z)

The merge-vs-padding interaction: the docstring of `_merge_overlapping` and the module say the goal is "never replay the same footage." But padding is applied AFTER the merge, and two distinct (gapped) rallies separated by less than `begin_padding + end_padding` will produce overlapping padded clips ? replayed footage. That's a design tradeoff, not necessarily a bug (padding is meant to be small). I'll note it but it's low-confidence as a "defect" since it's arguably acceptable behavior. Let me focus on more concrete defects.

Let me re-examine `compiler.py` lines 296-321 ? the retry/produced logic carefully. There's a subtle bug path. Let me trace:

- `attempts = [vf_args, []]` if watermark, else `[[]]`.
- Loop: `attempt_vf = vf_args` (first). Run ffmpeg. If `CalledProcessError`: set `last_exc`, if `attempt_vf and vf_args` ? `continue` (retry without).
- Second iteration `attempt_vf = []`: run ffmpeg. If it succeeds (no exception), check `os.path.exists(clip_path) and getsize > 0` ? `produced = True`.
- If the file is empty/missing: `last_exc = None; break`.

Now consider: first attempt succeeds at subprocess level (no exception) but produces empty file. Then `os.path.exists()` check fails ? `last_exc = None; break`. So `produced` stays False, `last_exc` is None ? falls to the `else` branch "exited successfully but output clip missing." Good.

But there's a real subtle issue. Consider watermark enabled, first attempt (with vf) raises CalledProcessError but the PARTIAL output file `clip_path` was created by ffmpeg (ffmpeg often creates the output file even on failure). The retry without vf runs `subprocess.run` ? but wait, the retry could ALSO fail. Let me trace the dangerous case: first attempt fails AND leaves a non-empty partial file, second attempt fails too. After second `CalledProcessError`: `attempt_vf = []` which is falsy ? `if attempt_vf and vf_args` is False ? `break`. `produced=False`, `last_exc` set ? error branch. Fine, partial file isn't appended. OK.

Let me look at the gemini.py segmenter and its `_parse_time` (referenced as the mirror), plus check `extract_video_chunk` contract.

35. user (2026-06-18T07:48:59.612Z)

```
1       import os
2       import time
3       import logging
4       import subprocess
5       import concurrent.futures
6       from typing import List, Dict, Any, Tuple
7       from backend.pipeline.segmenters.base import BasePlaySegmenter, segmenter_registry
8       from backend.utils.video import get_video_duration, extract_video_chunk
9       from backend.sports import sport_registry
10      from backend.providers import provider_registry
11      from backend.reporting import run_signals
12
13      logger = logging.getLogger(__name__)
14
```

```
15
16      def _chunk_workdir(video_id: str) -> str:
17          """Per-video chunk staging dir under output/chunks/<video_id>.
18
19          Concurrency isolation: two runs for DIFFERENT videos must not share one chunk dir ?
20          otherwise one run finishing (and rmdir-ing the shared dir) can undo the other's
21          in-flight work. A per-video subdir means each run only ever creates/cleans its own.
22          """
23          return os.path.join("output", "chunks", video_id)
24
25
26      class GeminiPlaySegmenter(BasePlaySegmenter):
27          @property
28          def name(self) -> str:
29              return "gemini"
30
31          @property
32          def description(self) -> str:
33              return "High-precision AI-powered semantic play segment detection using Google
Gemini Multimodal API."
34
35          def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None, max_frames: int = None) -> List[Dict[str, Any]]:
36              """
37              Processes a video to detect play segments using Google Gemini Multimodal Video
API,
38              populates SQLite, and indexes detailed events.
39              """
40              # Get sport profile config
41              sport_name = self.config.get("sport", "badminton")
42              try:
43                  sport_profile = sport_registry.get(sport_name)
44              except KeyError as e:
45                  logger.error(str(e))
46                  return []
47
48              # Get AI provider and model config
49              idx_cfg = self.config.get("indexing", {})
50              provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
51              model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
"gemini-2.5-flash"))
52
53              # Get appropriate API key based on the provider
54              api_key_env_var = f"{provider_name.upper()}_API_KEY"
55              api_key = os.environ.get(api_key_env_var)
56              if not api_key:
57                  logger.error(
58                      f"{api_key_env_var} environment variable is not set! "
59                      f"To run the {provider_name} segmenter, set your API key. "
60                      f"In PowerShell: $env:{api_key_env_var}=\"your_api_key_here\""
61                  )
62                  return []
63
64              try:
65                  provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
66              except KeyError as e:
67                  logger.error(str(e))
68                  return []
69
70              # A: Check for debug_duration to trim the video first
71              debug_duration = self.config.get("indexing", {}).get("debug_duration")
72              video_to_upload = video_path
73              is_trimmed = False
```

```
74              temp_trimmed_path = None
75
76              if debug_duration:
77                  logger.info(f"DEBUG FEATURE: Trimming video to first {debug_duration}
seconds for fast processing...")
78                  temp_trimmed_path = os.path.join("output", f"temp_trimmed_{video_id}.mp4")
79                  os.makedirs("output", exist_ok=True)
80                  if os.path.exists(temp_trimmed_path):
81                      try:
82                          os.remove(temp_trimmed_path)
83                      except Exception:
84                          pass
85
86                  cmd = [
87                      "ffmpeg", "-y",
88                      "-ss", "0",
89                      "-t", str(debug_duration),
90                      "-i", video_path,
91                      "-c", "copy",
92                      temp_trimmed_path
93                  ]
94                  logger.debug(f"Running command: {' '.join(cmd)}")
95                  try:
96                      subprocess.run(cmd, check=True, stdout=subprocess.DEVNULL,
stderr=subprocess.DEVNULL)
97                      logger.info(f"Trimmed video successfully created at:
{temp_trimmed_path}")
98                      video_to_upload = temp_trimmed_path
99                      is_trimmed = True
100                 except Exception as e:
101                     logger.warning(f"Failed to trim video using ffmpeg: {e}. Falling back to
full video.")
102
103
104
105             # Helper to clean up local temp files
106             def cleanup_temp_files():
107                 if is_trimmed and temp_trimmed_path and os.path.exists(temp_trimmed_path):
108                     try:
109                         os.remove(temp_trimmed_path)
110                     except Exception:
111                         pass
112
113             # 1. Determine actual video duration
114             video_duration = get_video_duration(video_to_upload)
115             if video_duration > 0:
116                 logger.info(f"Detected video duration: {video_duration:.1f}s")
117             else:
118                 logger.warning("Could not detect video duration. Prompt will not include
duration context.")
119
120             # 2. Decide whether to use chunked processing
121             idx_cfg = self.config.get("indexing", {})
122             chunk_duration = idx_cfg.get("chunk_duration", 300)    # 5 minutes default
123             chunk_overlap = idx_cfg.get("chunk_overlap", 30)       # 30 seconds overlap
124             use_chunking = video_duration > 0 and video_duration > chunk_duration
125             # Re-encode + downscale chunks for upload (0 = legacy stream-copy). Stream-
copied
126             # 4K chunks (~1.3GB/90s) exceed the provider's MAX_UPLOAD_MB and are refused, so
127             # every chunk of a 4K source gets skipped; Gemini analyzes at low res anyway.
128             ai_scale_width = int(idx_cfg.get("ai_chunk_scale_width", 1280) or 0)
129
130             ai_segments = []
131             # Per-video chunk workdir (concurrency isolation): a concurrent run for another
132             # video keeps its own dir, so cleanup here never removes another run's chunks.
```

```
            chunks_dir = _chunk_workdir(video_id)

            if use_chunking:
                # --- CHUNKED PROCESSING ---
                step = chunk_duration - chunk_overlap
                chunk_starts = []
                t = 0.0
                while t < video_duration:
                    chunk_starts.append(t)
                    t += step
                num_chunks = len(chunk_starts)
                logger.info(
                    f"Video is {video_duration:.1f}s ? splitting into {num_chunks}
overlapping chunks "
                    f"({chunk_duration}s each, {chunk_overlap}s overlap)."
                )

                os.makedirs(chunks_dir, exist_ok=True)

                def process_single_chunk(ci: int, chunk_start: float) -> Tuple[str, list,
dict]:
                    chunk_end = min(chunk_start + chunk_duration, video_duration)
                    actual_chunk_dur = chunk_end - chunk_start
                    chunk_label = f"Chunk {ci+1}/{num_chunks}"
                    chunk_path = os.path.join(chunks_dir, f"chunk_{video_id}_{ci}.mp4")

                    # Extract chunk with ffmpeg
                    logger.info(f"Extracting {chunk_label}: {chunk_start:.1f}s -
{chunk_end:.1f}s ({actual_chunk_dur:.1f}s)")
                    success = extract_video_chunk(
                        video_to_upload, chunk_start, actual_chunk_dur, chunk_path,
                        reencode=ai_scale_width > 0,
                        scale_width=ai_scale_width if ai_scale_width > 0 else None,
                    )
                    if not success:
                        logger.error(f"Failed to extract {chunk_label}. Skipping.")
                        return chunk_label, [], {}

                    # Measure chunk size
                    try:
                        chunk_size_mb = os.path.getsize(chunk_path) / (1024 * 1024)
                    except Exception:
                        chunk_size_mb = 0.0

                    # Get prompt from sport profile
                    prompt = sport_profile.get_prompt(chunk_duration=actual_chunk_dur)

                    # Process this chunk through the provider
                    chunk_segments, metrics = provider.analyze_video(
                        chunk_path, prompt, chunk_duration=actual_chunk_dur,
label=chunk_label
                    )
                    metrics["file_size_mb"] = chunk_size_mb
                    metrics["segments_found"] = len(chunk_segments)

                    # Offset timestamps back to full-video coordinates
                    for segment in chunk_segments:
                        if "start_time" in segment:
                            try:
                                segment["start_time"] = float(segment["start_time"]) +
chunk_start
                            except (ValueError, TypeError):
                                pass
                        if "end_time" in segment:
                            try:
```

```
                                segment["end_time"] = float(segment["end_time"]) +
chunk_start
                            except (ValueError, TypeError):
                                pass

                    logger.info(f"{chunk_label} returned {len(chunk_segments)} play
segments.")

                    # Clean up chunk file
                    try:
                        os.remove(chunk_path)
                    except Exception:
                        pass

                    return chunk_label, chunk_segments, metrics

                # Run chunks in parallel. Workers come from indexing.ai_max_workers (same
knob
                # the hybrids honor; was hardcoded 5). ai_max_workers=1 = serial ? needed to
                # gently ramp a cold/prepaid Gemini project, whose burst throttling returns
a
                # misleading "credits depleted" 429 under 5 simultaneous uploads.
                all_metrics = {}
                max_workers = max(1, int(self.config.get("indexing",
{}).get("ai_max_workers", 5)))
                logger.info("Starting parallel processing with up to %d concurrent
workers...", max_workers)
                t_chunking_start = time.time()
                with concurrent.futures.ThreadPoolExecutor(max_workers=max_workers) as
executor:
                    futures = [executor.submit(process_single_chunk, ci, start) for ci,
start in enumerate(chunk_starts)]
                    for future in concurrent.futures.as_completed(futures):
                        label, segments, metrics = future.result()
                        ai_segments.extend(segments)
                        if metrics:
                            all_metrics[label] = metrics

                # Oversize refusals must be LOUD: each one is a time range with NO analysis,
                # so "0 segments / success" here would be a silent failure, not "no
rallies".
                oversize_skips = sum(1 for m in all_metrics.values() if
m.get("oversize_skipped"))
                if oversize_skips:
                    logger.error(
                        f"{oversize_skips}/{num_chunks} chunks exceeded the provider upload
cap and were "
                        f"SKIPPED WITHOUT ANALYSIS ? segments are missing for those time
ranges. "
                        f"Set indexing.ai_chunk_scale_width (default 1280) so chunks are re-
encoded/"
                        f"downscaled before upload."
                    )
                    run_signals.incr(video_id, "oversize_clips_skipped", oversize_skips)

                # Print Summary Log
                summary_lines = [
                    "",
                    "=" * 60,
                    "               GEMINI CHUNKING SUMMARY",
                    "=" * 60,
                    f" Total time: {time.time() - t_chunking_start:.1f}s",
                    f" Total chunks: {num_chunks}",
                    f" Total play segments found: {len(ai_segments)}",
                    "-" * 60,
```

```
                    f" {'Chunk':<12} | {'Size':<8} | {'Upload':<8} | {'Process':<8} |
{'Gen.':<8} | {'Segments'}",
                    "-" * 60,
                ]
                for i in range(num_chunks):
                    lbl = f"Chunk {i+1}/{num_chunks}"
                    m = all_metrics.get(lbl, {})
                    sz = f"{m.get('file_size_mb', 0):.1f}MB" if m else "N/A"
                    upl = f"{m.get('upload_time', 0):.1f}s" if m else "N/A"
                    prc = f"{m.get('process_time', 0):.1f}s" if m else "N/A"
                    gen = f"{m.get('gen_time', 0):.1f}s" if m else "N/A"
                    seg = m.get('segments_found', 0) if m else 0
                    summary_lines.append(f" {lbl:<12} | {sz:<8} | {upl:<8} | {prc:<8} |
{gen:<8} | {seg}")
                summary_lines.append("=" * 60)
                logger.info("\n".join(summary_lines))

                # Clean up local temp files
                cleanup_temp_files()
                try:
                    os.rmdir(chunks_dir)
                except Exception:
                    pass

                logger.info(f"All chunks processed. {len(ai_segments)} total play segments
before deduplication.")

            else:
                # --- SINGLE-FILE PROCESSING (short videos) ---
                try:
                    file_sz = os.path.getsize(video_to_upload) / (1024 * 1024)
                except Exception:
                    file_sz = 0.0

                # Get prompt from sport profile
                prompt = sport_profile.get_prompt(chunk_duration=video_duration)
                ai_segments, metrics = provider.analyze_video(
                    video_to_upload, prompt, chunk_duration=video_duration,
label="SingleFile"
                )
                if metrics.get("oversize_skipped"):
                    logger.error(
                        f"The video ({file_sz:.0f}MB) exceeded the provider upload cap and
was "
                        f"SKIPPED WITHOUT ANALYSIS ? 0 segments here does NOT mean 'no
rallies'."
                    )
                    run_signals.incr(video_id, "oversize_clips_skipped", 1)
                summary_lines = [
                    "",
                    "=" * 50,
                    "        GEMINI SINGLE FILE SUMMARY",
                    "=" * 50,
                    f" File Size: {file_sz:.1f} MB",
                    f" Upload time: {metrics.get('upload_time', 0):.1f}s",
                    f" API Processing time: {metrics.get('process_time', 0):.1f}s",
                    f" Prompt Generation time: {metrics.get('gen_time', 0):.1f}s",
                    f" Play segments found: {len(ai_segments)}",
                    "=" * 50,
                ]
                logger.info("\n".join(summary_lines))
                cleanup_temp_files()

            if not ai_segments:
                return []
```

```
304
305             # Populate SQLite DB with parsed play segments
306             segments_data = []
307             logger.info(f"Processing {len(ai_segments)} play segments through
validation...")
308
309             def parse_time(val) -> float:
310                 """Converts a timestamp value to total float seconds.
311                 Handles float/int pass-through, plain numeric strings, and
312                 colon-separated formats (MM:SS, HH:MM:SS). Logs warnings
313                 when unexpected formats are encountered."""
314                 if isinstance(val, (int, float)):
315                     return float(val)
316                 if isinstance(val, str):
317                     val = val.strip()
318                     if ":" in val:
319                         # Gemini returned MM:SS or HH:MM:SS despite being asked for decimal
seconds.
320                         # Convert it, but warn so the issue is visible in logs.
321                         parts = val.split(":")
322                         parts.reverse()
323                         total = 0.0
324                         for i, p in enumerate(parts):
325                             try:
326                                 total += float(p) * (60 ** i)
327                             except ValueError:
328                                 pass
329                         logger.warning(f"Timestamp '{val}' was in colon-separated format
(MM:SS/HH:MM:SS) instead of decimal seconds. Converted to {total:.2f}s.")
330                         return total
331                     try:
332                         return float(val)
333                     except ValueError:
334                         logger.warning(f"Could not parse timestamp string '{val}' as a
number. Defaulting to 0.0.")
335                         return 0.0
336                 logger.warning(f"Unexpected timestamp type {type(val).__name__} for value
'{val}'. Defaulting to 0.0.")
337                 return 0.0
338
339             # 7. Parse timestamps and build candidate segment list
340             idx_cfg = self.config.get("indexing", {})
341             min_segment_duration = idx_cfg.get("min_segment_duration", 3.0)
342             max_segment_duration = 90.0  # Singles play segments almost never exceed this
343
344             candidates = []
345             for idx, segment in enumerate(ai_segments):
346                 start = parse_time(segment.get("start_time", 0.0))
347                 end = parse_time(segment.get("end_time", 0.0))
348                 if start >= end:
349                     logger.info(f"Skipping segment #{idx+1}: start_time ({start:.2f}s) >=
end_time ({end:.2f}s)")
350                     continue
351                 candidates.append({
352                     "idx": idx,
353                     "start": start,
354                     "end": end,
355                     "raw": segment
356                 })
357
358             # 8. Post-hoc validation pass
359             logger.info(f"Running validation on {len(candidates)} candidate play
segments...")
360             validated = []
361             rejected = []
```

```
362
363             for c in candidates:
364                 label = f"Segment #{c['idx']+1} [{c['start']:.2f}s - {c['end']:.2f}s]"
365
366                 # A: Reject segments that start beyond the video
367                 if video_duration > 0 and c["start"] >= video_duration:
368                     rejected.append((label, f"start_time ({c['start']:.2f}s) is beyond video
duration ({video_duration:.1f}s)"))
369                     continue
370
371                 # B: Clamp end_time to video duration
372                 if video_duration > 0 and c["end"] > video_duration:
373                     logger.info(f"{label}: end_time ({c['end']:.2f}s) exceeds video duration
({video_duration:.1f}s). Clamping.")
374                     c["end"] = video_duration
375
376                 dur = c["end"] - c["start"]
377
378                 # C: Reject segments shorter than min_segment_duration
379                 if dur < min_segment_duration:
380                     rejected.append((label, f"duration ({dur:.2f}s) is below minimum
({min_segment_duration}s)"))
381                     continue
382
383                 # D: Flag suspiciously long segments
384                 if dur > max_segment_duration:
385                     logger.warning(
386                         f"{label}: duration ({dur:.1f}s) is unusually long
(>{max_segment_duration}s). "
387                         f"This may be a detection error. Keeping but flagging."
388                     )
389
390                 validated.append(c)
391
392             # E: Resolve overlapping segments (keep the longer one)
393             if len(validated) > 1:
394                 # Sort by start time
395                 validated.sort(key=lambda r: r["start"])
396                 non_overlapping = [validated[0]]
397                 for curr in validated[1:]:
398                     prev = non_overlapping[-1]
399                     # Check if current segment overlaps with the previous one
400                     if curr["start"] < prev["end"]:
401                         prev_dur = prev["end"] - prev["start"]
402                         curr_dur = curr["end"] - curr["start"]
403                         kept, dropped = (prev, curr) if prev_dur >= curr_dur else (curr,
prev)
404                         kept_label = f"Segment #{kept['idx']+1} [{kept['start']:.2f}s -
{kept['end']:.2f}s]"
405                         dropped_label = f"Segment #{dropped['idx']+1}
[{dropped['start']:.2f}s - {dropped['end']:.2f}s]"
406                         rejected.append((dropped_label, f"overlaps with {kept_label} ? kept
the longer segment"))
407                         if kept == curr:
408                             non_overlapping[-1] = curr  # Replace previous with current
409                     else:
410                         non_overlapping.append(curr)
411                 validated = non_overlapping
412
413             # Log validation summary
414             if rejected:
415                 reject_lines = [f"  ? {label}: {reason}" for label, reason in rejected]
416                 logger.info(f"{len(rejected)} play segments were REJECTED:\n" +
"\n".join(reject_lines))
417             logger.info(f"{len(validated)} play segments passed validation (out of
```

```
{len(ai_segments)} from Gemini).")
418
419             stitch_cfg = self.config.get("stitching", {})
420             begin_pad = stitch_cfg.get("begin_padding", 0.5)
421             end_pad = stitch_cfg.get("end_padding", 0.5)
422
423             total_highlight_dur = 0.0
424             for c in validated:
425                 total_highlight_dur += (c["end"] - c["start"]) + begin_pad + end_pad
426
427             mins = int(total_highlight_dur // 60)
428             secs = int(total_highlight_dur % 60)
429             logger.info(
430                 f"Estimated final highlight video duration: {mins:02d}:{secs:02d} "
431                 f"({total_highlight_dur:.1f}s total; includes {begin_pad}s begin +
{end_pad}s end padding per segment)"
432             )
433
434             # 9. Populate SQLite DB with validated play segments
435             for c in validated:
436                 start = c["start"]
437                 end = c["end"]
438                 segment = c["raw"]
439                 dur = end - start
440
441                 # Extract shot_count from Gemini or compute estimate if not present
442                 shot_count = segment.get("shuttle_exchange_count")
443                 if shot_count is None:
444                     shot_count = max(3, int(dur / 0.8))  # 1 shot every 0.8 seconds as
fallback estimate
445                 else:
446                     try:
447                         shot_count = int(shot_count)
448                     except Exception:
449                         shot_count = max(3, int(dur / 0.8))
450
451                 # Only store metrics we actually know ? no fabricated data
452                 metadata = {
453                     "shot_count": shot_count,
454                     "shot_count_confidence": segment.get("shot_count_confidence",
"Unknown"),
455                     "detector": model_name
456                 }
457
458                 segment_id = self.db.add_play_segment(video_id, start, end, None, None,
metadata)
459
460                 if segment_id:
461                     segments_data.append({
462                         "id": segment_id,
463                         "start_time": start,
464                         "end_time": end,
465                         "duration": dur,
466                         "metadata": metadata
467                     })
468
469             return segments_data
470
471     # Register the segmenter
472     segmenter_registry.register(GeminiPlaySegmenter)
473
```

36. assistant (2026-06-18T07:49:17.568Z)

I found a real bug in `gemini.py`. In the overlap-resolution loop (lines 392-411), when `kept ==

curr` (the current segment is longer and replaces the previous), the loop sets `non_overlapping[-1] = curr`. But consider three segments A, B, C where A overlaps B (B longer, kept), then C is compared against `non_overlapping[-1]` which is now B. This is correct chaining.

However, the `kept == curr` / `kept == prev` comparison uses `==` on dicts. Two distinct candidate dicts could compare equal if they have identical contents. That's an edge case. More importantly, let me look at a subtler issue: when `prev` is kept (prev_dur >= curr_dur), `curr` is dropped and NOT appended ? correct. When `curr` is kept, `non_overlapping[-1] = curr`. But what if curr overlaps prev AND the next segment doesn't overlap curr but DID overlap prev's original extent? Since curr is shorter-or-equal end... actually curr could have a smaller end than prev. If curr replaces prev but curr.end < prev.end, a subsequent segment that started within [curr.end, prev.end) would now be treated as non-overlapping. That's a logic imperfection but not a crash.

Let me focus. The `kept == curr` dict comparison: `kept` and `dropped` are assigned from `(prev, curr)`. `kept == curr` ? comparing by value. If `prev` and `curr` happen to be value-equal dicts (e.g., identical duplicate segments), `prev_dur >= curr_dur` is True so `kept = prev`, then `kept == curr` evaluates True by value-equality even though we meant `kept is prev`. Then `non_overlapping[-1] = curr` runs ? harmless here since they're equal. Low impact.

Let me check `extract_video_chunk` return contract and the `w_dur` zero/negative edge case in handoff. If `window["start"]` is near `video_duration` and end too, `w_start = max(0, start-pad)`, `w_end = min(video_duration, end+pad)`. If start > video_duration somehow, `w_end` could be < `w_start` ? negative `w_dur`. But windows come from trajectory within the video. Let me verify there's no place a candidate window's end can exceed video_duration causing `w_dur` issues. More productive: let me check `_get_video_duration` in compiler and the `_parse_time` colon logic for a real bug.

In `_parse_time` (compiler line 61-71): for `"1:30"`, parts=["1","30"], reversed=["30","1"], total = 30*1 + 1*60 = 90. Correct. For negative like `"-1:30"`? parts=["-1","30"], gives 30 + (-1)*60 = -30. Edge case, not a real trigger.

Let me look at the motion.py smash event generation again - the `random.uniform(start + 1.5, end - 1.0)` with very short segments. Min segment duration default is 3.0s. So `start+1.5` to `end-1.0` where `end-start >= 3.0` means `end-1.0 >= start+2.0 > start+1.5`. So bounds are valid when min_segment_duration >= 3.0. But `min_segment_duration` is configurable and could be set < 2.5. If a segment is exactly e.g. 2.0s long, `start+1.5 > end-1.0` (start+1.5 vs start+1.0), reversed bounds. `random.uniform` handles reversed bounds fine (returns value in range). Not a crash.

Let me check the `_axis_estimate` caching in fusion using `id(points)` ? a real concern. Let me look at how `rally_gate.estimate_play_axis` can return, and whether `assert est is not None` can fire.

37. user (2026-06-18T07:49:18.147Z)

```
1       """Rally-start gate: suppress between-point false fires using shuttle net-crossings.
2
3       WASB (shuttle-only) over-detects: when a player flicks/tosses the shuttle back to
4       the opponent for service after losing a point, that single trajectory looks like a
5       rally. The discriminator is structural, not duration-based: a real rally has the
6       shuttle crossing the net MULTIPLE times (each shot lands on the other side); a
7       single service feed crosses ~once.
8
9       This module is camera-agnostic and needs NO new model ? it works on the trajectory
10      we already produce (Frame,Visibility,X,Y). It estimates the play axis (the image
11      axis with the larger shuttle spread) and the net position (median shuttle coord on
12      that axis ? the shuttle clusters at the two ends and crosses the net region often,
13      so the median sits near the net), then counts sign changes of (coord - net) across
14      the visible points inside a candidate window, with a deadband to ignore jitter.
15
16      Gate A in the over-fire fix spectrum (B = player-stance state machine, future).
17      Pure functions only ? no I/O, unit-tested; intended as a post-filter on the
18      windows from trajectory_to_action_windows, or to fold into it later.
```

```
    """
    from __future__ import annotations

    from dataclasses import dataclass
    from typing import List, Optional, Sequence, Tuple

    Interval = Tuple[float, float]


    @dataclass
    class GatedWindow:
        start: float
        end: float
        crossings: int
        visible_points: int
        kept: bool


    def _spread(vals: Sequence[float]) -> float:
        """Robust spread = p95 - p5 (ignores a few outlier detections)."""
        if len(vals) < 2:
            return 0.0
        s = sorted(vals)
        lo = s[max(0, int(0.05 * (len(s) - 1)))]
        hi = s[min(len(s) - 1, int(0.95 * (len(s) - 1)))]
        return hi - lo


    def _median(vals: Sequence[float]) -> float:
        if not vals:
            return 0.0
        s = sorted(vals)
        n = len(s)
        return s[n // 2] if n % 2 else 0.5 * (s[n // 2 - 1] + s[n // 2])


    def estimate_play_axis(points) -> Tuple[str, float, float]:
        """Return (axis 'x'|'y', net_value, span) from visible points.

        Play axis = the image axis along which the shuttle travels most (larger spread).
        For a baseline camera that's Y (players top/bottom); for a side camera, X.
        """
        xs = [p.x for p in points if p.visible]
        ys = [p.y for p in points if p.visible]
        sx, sy = _spread(xs), _spread(ys)
        if sx >= sy:
            return "x", _median(xs), sx
        return "y", _median(ys), sy


    def count_net_crossings(window_points, axis: str, net: float, deadband: float) -> int:
        """Count sign changes of (coord - net) across visible points, ignoring the
        deadband zone near the net (so jitter at the net doesn't inflate the count)."""
        crossings = 0
        last_side = None
        for p in window_points:
            if not p.visible:
                continue
            v = (p.x if axis == "x" else p.y) - net
            if abs(v) < deadband:
                continue
            side = v > 0
            if last_side is not None and side != last_side:
                crossings += 1
            last_side = side
```

```
84          return crossings
85
86
87      def _points_in(points, fps: float, start: float, end: float):
88          return [p for p in points if start * fps <= p.frame <= end * fps]
89
90
91      def gate_windows(points, windows: List[Interval], fps: float,
92                       min_crossings: int = 2, deadband_frac: float = 0.06,
93                       estimate: Optional[Tuple[str, float, float]] = None) ->
List[GatedWindow]:
94          """Annotate each (start,end) window with its shuttle net-crossing count and a
95          keep decision (crossings >= min_crossings). Deadband is a fraction of the play
96          axis span. Net/axis are estimated once over the whole trajectory (stable).
97
98          `estimate` lets a caller scoring MANY windows off one trajectory pass a precomputed
99          `(axis, net, span)` (from `estimate_play_axis`) so the O(P log P) estimate isn't
redone
100         per window ? same result, avoids the per-window re-estimation."""
101         axis, net, span = estimate if estimate is not None else estimate_play_axis(points)
102         deadband = deadband_frac * span
103         out: List[GatedWindow] = []
104         for (s, e) in windows:
105             wp = _points_in(points, fps, s, e)
106             c = count_net_crossings(wp, axis, net, deadband)
107             vis = sum(1 for p in wp if p.visible)
108             out.append(GatedWindow(start=s, end=e, crossings=c, visible_points=vis,
109                                    kept=c >= min_crossings))
110         return out
111
112
113     def apply_gate(points, windows: List[Interval], fps: float,
114                    min_crossings: int = 2, deadband_frac: float = 0.06) -> List[Interval]:
115         """Convenience: return only the windows that pass the crossing gate."""
116         return [(g.start, g.end) for g in
117                 gate_windows(points, windows, fps, min_crossings, deadband_frac) if g.kept]
118
```

38. assistant (2026-06-18T07:49:41.427Z)

`estimate_play_axis` always returns a tuple (never None), so the `assert est is not None` in fusion can't fire. Fine.

Now, the `id(points)` cache key in fusion (`_axis_estimate`). `points` is the whole-video trajectory list, computed once in `process_video` and passed to every `_enrich_candidate_stats` call. Since it's the same list object for the whole video, `id(points)` is stable within one `process_video` call. The cache is per-instance. Across two `process_video` calls on the same instance, a new `points` list is created, so `id()` differs (unless GC reuses the address ? but the old list is still referenced via closure during the loop, and process_video doesn't retain it after). Actually there's a classic `id()` reuse hazard: if list A is freed and list B is allocated at the same address, `id(B) == id(A)` and the stale cache for A would be wrongly reused for B. But within a single process_video, points stays alive the whole time, so no reuse mid-call. Between calls, the first enrich call re-estimates if `_axis_cache_key != key`. If by bad luck `id` matches a freed previous points, it returns stale axis. This is a real but extremely-low-probability latent bug, and these are per-video so axis would be similar. Low confidence; I'll note it lightly or skip.

Let me look more carefully at a concrete reachable defect. Back to `compiler.py` `_get_video_duration`: `subprocess.run(..., check=True)` then `float(result.stdout.strip())`. If ffprobe returns "N/A" (happens for some streams), `float("N/A")` raises ValueError, caught by `except Exception` ? returns 0.0. Fine.

Let me check the YOLO `_merge_stats` ? when merging two overlapping windows, it takes `start = a["start"]` and `end = max(a["end"], b["end"])`. But it does NOT carry forward the merged window if a THIRD window `c` overlaps the merged one. Let me re-trace lines 223-232:

```
merged_windows = [all_candidate_windows[0]]
for current in all_candidate_windows[1:]:
    last = merged_windows[-1]
    if current["start"] <= last["end"]:
        merged_windows[-1] = _merge_stats(last, current)
    else:
        merged_windows.append(current)
```

This correctly chains. `_merge_stats` returns a dict but is it missing keys that downstream code reads? Downstream reads `peak_velocity`, `mean_velocity`, `active_frame_count`, `track_id_count`, `start`, `end`, `chunk_index`. `_merge_stats` returns exactly those 7. Good.

Now let me examine the most promising lead carefully: in `trajectory_hybrid.py`, the candidate persistence vs handoff. `candidate_ids[wi]` is indexed by `wi` which comes from `handoff_indices` = list of indices into `all_candidate_windows`. `candidate_ids` has length `len(all_candidate_windows)`. Consistent. Good.

Let me verify the `skip_ai` path: when `store_candidates` is False, `candidate_ids` is all `None`. In skip_ai branch, `_candidate_id: candidate_ids[wi]` = None. Then Phase 4 `link_candidate_to_segment` guarded by `if candidate_id is not None`. Fine.

Let me check `_select_for_handoff` for the base `process_video` when `all_candidate_windows` is non-empty but `handoff_indices` returns empty (base returns all, fusion has never-empty guard). For base it's always all. Fine.

Let me look at the scene_cut validator's `cuts_per_minute` math ? `sampled_duration = (sampled_count * sample_interval) / fps`. But sampling stops at `sampled_count > 100` (so up to 101 frames). The `frame_idx += sample_interval` and break when `frame_idx >= total_frames or sampled_count > 100`. With `cap.set(POS_FRAMES, frame_idx)` each iteration ? this is correct but slow. The math: if it samples 101 frames at interval, `sampled_duration` could exceed real video duration if total_frames < 101*interval... no, it breaks at total_frames. Fine.

Let me now look at the blur validator carefully for a real off-by-one. `step = max(1, total_frames // num_samples)`, loops `i in range(num_samples)`, `frame_idx = i * step`. Max `frame_idx = 14 * step`. With `step = total_frames//15`, `14 * (total_frames//15)` ? total_frames. Fine, the `if not ret: continue` handles overshoot.

Let me look for the real defect more systematically. Let me check `extract_video_chunk` and `get_video_duration` signatures.

39. user (2026-06-18T07:49:42.293Z)

```
1       import subprocess
2       import logging
3       from typing import Optional
4
5       logger = logging.getLogger(__name__)
6
7       def get_video_duration(video_path: str) -> float:
8           """Returns the exact duration of a video file in seconds using ffprobe.
9           Returns 0.0 on failure."""
10          try:
11              duration_output = subprocess.check_output([
12                  "ffprobe", "-v", "error", "-show_entries", "format=duration",
13                  "-of", "default=noprint_wrappers=1:nokey=1", video_path
14              ]).decode().strip()
15              return float(duration_output)
16          except Exception as e:
17              logger.error(f"Could not detect video duration for {video_path}: {e}")
18              return 0.0
19
20      def extract_video_chunk(video_path: str, start_time: float, duration: float,
21                              output_path: str, reencode: bool = False,
```

```
22                              scale_width: Optional[int] = None) -> bool:
23          """Extracts a chunk from a video file.
24
25          Args:
26              reencode: If True, re-encodes with libx264 (slower but frame-accurate).
27                        If False, uses stream-copy (fast but cuts at nearest keyframe).
28                        Use True for short clips where keyframe drift matters.
29              scale_width: If set (requires reencode), downscale to this width (height auto,
30                        even). Useful for high-res sources headed to an API that analyses at
31                        low res anyway ? shrinks uploads with no analysis loss.
32          """
33          cmd = [
34              "ffmpeg", "-y",
35              "-ss", str(start_time),
36              "-t", str(duration),
37              "-i", video_path,
38          ]
39          if reencode:
40              if scale_width:
41                  cmd += ["-vf", f"scale={int(scale_width)}:-2"]
42              cmd += ["-c:v", "libx264", "-preset", "ultrafast", "-crf", "23", "-c:a", "aac"]
43          else:
44              cmd += ["-c", "copy"]
45          cmd.append(output_path)
46          try:
47              subprocess.run(cmd, check=True, stdout=subprocess.DEVNULL,
stderr=subprocess.DEVNULL)
48              return True
49          except Exception as e:
50              logger.error(f"Failed to extract chunk {start_time}s - {start_time+duration}s:
{e}")
51              return False
52
```

40. assistant (2026-06-18T07:49:59.907Z)

Now let me reconsider the gemini overlap-resolution bug more carefully ? the `kept == curr` dict comparison. Actually let me re-examine a cleaner reachable defect there. When `kept == curr` is checked with `==`, if `prev` and `curr` are value-equal dicts (which happens with exact-duplicate segments ? same idx, same start/end/raw), then `prev_dur >= curr_dur` is True, so `kept, dropped = (prev, curr)`. Then `if kept == curr:` is True (value equality) even though logically prev was kept ? `non_overlapping[-1] = curr`. Both equal, harmless.

But could `curr` be a duplicate of `prev` with a DIFFERENT idx? The `raw` dict differs or idx differs, so `==` would be False. The realistic risk is minimal. Let me drop that.

Let me reconsider the padding-overlap concern in the compiler as it relates to the stated contract. Let me check the relevance validator's unused variable - line 64 `h, w = frame.shape[:2]` then resizes to fixed (320,240), `h,w` unused. Style, skip.

Let me look at one thing that stood out: in `scene_cut.py`, the histogram is computed on the FULL frame (`hist = cv2.calcHist([hsv],...)`). `prev_hist = hist` stores a reference. `cv2.normalize(hist, hist, ...)` normalizes in-place. Next iteration creates a NEW hist. OK no aliasing.

Let me re-examine the most concrete candidate: the **`_get_video_duration` clamp in compiler with the merge ordering**. Actually, let me reconsider whether there's an issue with `padded_end` overlap producing duplicated footage that contradicts the explicit "never replay" contract. The merge only collapses ranges that overlap/abut on RAW boundaries. After merge, begin/end padding (default 0.5 + 1.0 = up to 1.5s) is added. Two rallies with a raw gap of e.g. 1.0s won't be merged (gap > 0 so distinct), but padded_end of first = end+1.0 and padded_start of second = start-0.5. If gap is 1.0s, first extends 1.0s past its end, second starts 0.5s before its start ? they overlap by 0.5s ? 0.5s of footage replayed in the reel. The module docstring says "A highlight reel must never replay the same footage." This IS a contract violation, but it's an inherent consequence of independent per-clip padding and is arguably

accepted (padding is for context, slight overlap at boundaries is cosmetic). I'll report it as low-confidence/low-severity since the "never replay" claim is explicitly stated but the overlap is small and by-design-ish.

Let me instead verify a sharper bug. Let me re-examine the `next_progress` logic and ETA in motion.py line 99: `eta = elapsed / (frame_idx / total_frames) - elapsed if frame_idx else 0`. If `total_frames == 0`, but `progress_step` would be 0 then and this branch is guarded by `if progress_step and ...`. So total_frames>0 here. Fine.

Let me check whether `total_frames` can be 0 but the while-loop still runs and `motion_signals` is empty ? `smoothed` empty ? no segments. Fine.

Let me look at the `__init__.py` files to see if there's an import-order/registration issue.

41. user (2026-06-18T07:50:00.865Z)

```
1       from backend.pipeline.validators.base import registry, VideoValidator, ValidationResult
2       from backend.pipeline.validators.format import FormatValidator
3       from backend.pipeline.validators.resolution_fps import ResolutionFPSValidator
4       from backend.pipeline.validators.pts_continuity import PTSContinuityValidator
5       from backend.pipeline.validators.scene_cut import SceneCutDensityValidator
6       from backend.pipeline.validators.blur import BlurValidator
7       from backend.pipeline.validators.relevance import RelevanceValidator
8
9       # Register all default validators in sequence (dependencies first)
10      registry.register(FormatValidator())
11      registry.register(ResolutionFPSValidator())
12      registry.register(PTSContinuityValidator())
13      registry.register(SceneCutDensityValidator())
14      registry.register(BlurValidator())
15      registry.register(RelevanceValidator())
16
17      __all__ = ["registry", "VideoValidator", "ValidationResult"]
18
```

42. user (2026-06-18T07:50:01.133Z)

```
1       from backend.pipeline.segmenters.base import segmenter_registry, BasePlaySegmenter
2       from backend.pipeline.segmenters.motion import MotionPlaySegmenter
3       from backend.pipeline.segmenters.gemini import GeminiPlaySegmenter
4       from backend.pipeline.segmenters.yolo_hybrid import HybridYoloSegmenter
5       from backend.pipeline.segmenters.tracknet_hybrid import TrackNetHybridSegmenter
6       from backend.pipeline.segmenters.wasb_hybrid import WasbHybridSegmenter
7       from backend.pipeline.segmenters.fusion_hybrid import FusionSegmenter
8
9       __all__ = [
10          "segmenter_registry",
11          "BasePlaySegmenter",
12          "MotionPlaySegmenter",
13          "GeminiPlaySegmenter",
14          "HybridYoloSegmenter",
15          "TrackNetHybridSegmenter",
16          "WasbHybridSegmenter",
17          "FusionSegmenter",
18      ]
19
```

43. assistant (2026-06-18T07:50:19.843Z)

Now let me reconsider the FormatValidator ? ResolutionFPS context dependency. `ResolutionFPSValidator` reads `context.get("ffprobe_meta")` set by `FormatValidator`. Registration order ensures FormatValidator runs first. But `run_all_with_context` catches exceptions per-validator. If FormatValidator throws (and doesn't set context), ResolutionFPS returns a clean failure. Fine.

But there's a subtle issue: `FormatValidator` only sets `context["ffprobe_meta"] = info` AFTER checking `info.get("streams")`. If streams missing, it returns early WITHOUT setting context ? ResolutionFPS then says "metadata missing." Fine.

Now, `ResolutionFPSValidator` uses `meta["streams"][0]`. `FormatValidator`'s ffprobe uses `-select_streams v:0`, so streams[0] is the video stream. But `ResolutionFPS` is registered separately and relies on that selection. If a future validator re-probes without `-select_streams`, streams[0] might be audio. Not a current bug.

Let me now look hard at one thing in `relevance.py`: `min_court_ratio = rel_cfg.get(..., sport_rel_cfg.get("court_pixels_min_ratio", 0.05))`. If config provides a string, comparison `avg_green_ratio < min_court_ratio` could TypeError. But config is typed/Pydantic per CLAUDE.md... but here config is a plain dict passed to validators. Low concern.

Let me verify the real candidate I'm most confident about. Let me re-examine `compiler.py` lines 315-321 closely for the "ffmpeg succeeded but produced nothing" + watermark retry path interaction ? there IS a subtle bug:

```python
for attempt_vf in attempts:
    cmd = base_cmd + attempt_vf + [clip_path]
    try:
        subprocess.run(cmd, capture_output=True, check=True)
    except subprocess.CalledProcessError as e:
        last_exc = e
        if attempt_vf and vf_args:
            ...
            continue
        break
    if os.path.exists(clip_path) and os.path.getsize(clip_path) > 0:
        produced = True
        ...
        break
    last_exc = None  # ffmpeg succeeded but produced nothing
    break
```

Consider watermark enabled (`vf_args` truthy, `attempts = [vf_args, []]`):
- Attempt 1 (`attempt_vf = vf_args`): subprocess SUCCEEDS (no exception) but produces an empty/missing file. Then the `if os.path.exists...` is False ? `last_exc = None; break`.

So we break out after the FIRST attempt with the watermark, WITHOUT retrying without the watermark, even though the watermark encode silently produced an empty file. The retry-without-watermark only triggers on a `CalledProcessError`, not on a silent empty-output. This means a watermark filter that makes ffmpeg exit 0 but emit nothing kills that segment entirely instead of falling back. That's a minor robustness gap, not a crash, and "exit 0 + empty output" is unusual. Lowish confidence.

Let me now look at a genuinely sharp issue. Let me re-examine the `_merge_overlapping` interaction with the `compile_highlights` re-validation. After `_merge_overlapping`, segments are clean `(float, float)` with `e > s`. Then in the loop, `_parse_time` is called AGAIN on already-float values (harmless), and `start >= end` re-checked (won't trigger post-merge), and `start < 0` clamped. But `_merge_overlapping` already dropped degenerate rows. So the per-iteration `start >= end` skip is dead for merged input. Not a bug.

Let me focus and check git log / tests for known patterns around these files to avoid re-reporting fixed issues, and look at `database.add_play_segment` return contract (the `if segment_id:` guard ? what if it returns 0?).

44. user (2026-06-18T07:50:21.116Z)

```
311:    def add_play_segment(self, video_id: str, start_time: float, end_time: float, server:
Optional[str] = None, receiver: Optional[str] = None, metadata: Optional[Dict[str, Any]] = None)
-> Optional[int]:
312-        try:
```

```
313-            with self._connect() as conn:
314-                cursor = conn.cursor()
315-                cursor.execute(
316-                    "INSERT INTO play_segments (video_id, start_time, end_time, duration,
server, receiver, metadata) VALUES (?, ?, ?, ?, ?, ?, ?)",
317-                    (video_id, start_time, end_time, end_time - start_time, server,
receiver, json.dumps(metadata or {}))
318-                )
319-                conn.commit()
320-                return cursor.lastrowid
321-        except Exception as e:
322-            logger.error(f"Failed to add play segment: {e}")
323-            return None
324-
325:    def add_event(self, segment_id: int, timestamp: float, event_type: str, player:
Optional[str] = None, details: Optional[Dict[str, Any]] = None) -> bool:
326-        try:
327-            with self._connect() as conn:
328-                conn.cursor().execute(
329-                    "INSERT INTO events (segment_id, timestamp, type, player, details)
VALUES (?, ?, ?, ?, ?)",
330-                    (segment_id, timestamp, event_type, player, json.dumps(details or {}))
331-                )
332-                conn.commit()
333-            return True
334-        except Exception as e:
335-            logger.error(f"Failed to add event: {e}")
336-            return False
337-
338:    def add_candidate_segment(self, video_id: str, detector: str, start_time: float,
end_time: float,
339-                              chunk_index: Optional[int] = None, confidence: Optional[float]
= None,
340-                              stats: Optional[Dict[str, Any]] = None,
341-                              detection_params: Optional[Dict[str, Any]] = None) ->
Optional[int]:
342-        """Persist one raw (pre-confirmation) detection. `stats`/`detection_params` are
343-        detector-specific dicts stored as JSON. Returns the new candidate id, or None."""
344-        try:
345-            with self._connect() as conn:
346-                cursor = conn.cursor()
347-                cursor.execute(
348-                    """INSERT INTO candidate_segments
349-                       (video_id, detector, chunk_index, start_time, end_time, duration,
350-                        confidence, stats, detection_params)
351-                       VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)""",
352-                    (video_id, detector, chunk_index, start_time, end_time, end_time -
start_time,
353-                     confidence, json.dumps(stats or {}), json.dumps(detection_params or
{}))
354-                )
355-                conn.commit()
356-                return cursor.lastrowid
357-        except Exception as e:
358-            logger.error(f"Failed to add candidate segment: {e}")
359-            return None
360-
361:    def link_candidate_to_segment(self, candidate_id: int, segment_id: int) -> bool:
362-        """Record that a candidate detection was confirmed as a given play_segment."""
363-        try:
364-            with self._connect() as conn:
365-                conn.cursor().execute(
366-                    "UPDATE candidate_segments SET confirmed_segment_id = ? WHERE id = ?",
367-                    (segment_id, candidate_id)
368-                )
```

```
369-                conn.commit()
370-            return True
371-        except Exception as e:
372-            logger.error(f"Failed to link candidate {candidate_id} to segment {segment_id}:
{e}")
373-            return False
374-
375-    def query_candidate_segments(self, video_id: str, detector: Optional[str] = None,
376-                                 only_unlabeled: bool = False) -> List[Dict[str, Any]]:
377-        """Fetch candidate detections for the annotation / fine-tuning workflow.
378-        `stats` and `detection_params` are deserialized from JSON."""
379-        query = "SELECT * FROM candidate_segments WHERE video_id = ?"
380-        params: List[Any] = [video_id]
381-
382-        if detector:
383-            query += " AND detector = ?"
384-            params.append(detector)
385-        if only_unlabeled:
386-            query += " AND human_label IS NULL"
```

45. assistant (2026-06-18T07:50:50.600Z)

`add_play_segment` returns `cursor.lastrowid`. `lastrowid` is never 0 for an INSERT (SQLite rowids start at 1), so `if segment_id:` is safe. Good.

Now let me reconsider the gemini overlap loop one more time for a REAL logic bug, because that's the validator/merge math the task highlights. Trace lines 392-411 with a concrete adversarial input. Three validated segments sorted by start:
- A = [0, 10] (dur 10)
- B = [5, 8] (dur 3) ? overlaps A, A longer ? keep A, drop B. `non_overlapping = [A]`.
- C = [9, 20] (dur 11) ? compare to `non_overlapping[-1] = A`. `C.start(9) < A.end(10)` ? overlap. `curr_dur(11) >= prev_dur(10)` ? keep C, drop A. `non_overlapping[-1] = C`. Result: [C].

But A was the only thing covering [0,5]. By dropping A in favor of C, we lose [0,9]. This is the documented "keep the longer one" heuristic ? lossy but intentional. Not a bug.

Now an adversarial case exposing an actual defect:
- A = [0, 100] (dur 100)
- B = [10, 20] (dur 10) ? overlaps A, keep A. [A]
- C = [30, 40] (dur 10) ? `C.start(30) < A.end(100)` overlap, keep A. [A]
- D = [50, 200] (dur 150) ? overlap with A, D longer, keep D. [D]

Fine. The chaining is correct because `non_overlapping[-1]` always holds the current "winner" and each new segment is compared against it. The only loss is when a kept segment has a smaller `end` than the dropped `prev` ? then a later segment between [kept.end, prev.end) is wrongly treated as non-overlapping. Example:
- A = [0, 100] (dur 100), kept first.
- B = [10, 105] (dur 95) ? overlaps A. `prev_dur(100) >= curr_dur(95)` ? keep A (end 100), drop B. [A]
- C = [102, 110] ? `C.start(102) < A.end(100)`? No, 102 >= 100 ? not overlap ? append C. [A, C].

But B (kept-rejected) extended to 105, and C starts at 102 < 105. In the original input C overlaps B's region, but since B was dropped, that's fine ? C doesn't overlap the KEPT set. Acceptable.

This loop is sound. Let me drop the gemini overlap angle.

Let me reconsider the strongest concrete finding. Let me re-examine the compiler's `temp_clips` ordering. Segments are sorted by `_merge_overlapping` (sorted ascending). The loop appends to `temp_clips` in segment order, and the concat list preserves order. Good ? reel is chronological.

Let me check one more: in `compiler.py`, the `cmd` variable used in the error message at line 328 (`print(f"  Command: {' '.join(cmd)}")`). `cmd` is set inside the for-attempt loop (line

301). After the loop, `cmd` holds the LAST attempt's command. If watermark retry happened, `cmd` is the no-watermark version ? minor logging inaccuracy. Not a defect.

Now let me look at whether there's an unguarded access. In `trajectory_hybrid.py` line 285: `segments, _ = provider.analyze_video(...)`. The provider contract returns a tuple. In `yolo_hybrid` line 291: `segments, metrics = provider.analyze_video(...)`. Both unpack 2. Consistent with gemini usage. Fine.

Let me check `process_candidate_window` in trajectory_hybrid more carefully ? `seg["start_time"] = float(seg["start_time"]) + w_start`. If `seg` lacks `start_time`, `KeyError` is caught. But it accesses `seg["end_time"]` on the next line ? if `start_time` exists but `end_time` doesn't, KeyError caught too. Both wrapped. Good. But note: it mutates `seg["start_time"]` THEN reads `seg["end_time"]` ? if end_time KeyErrors, start_time was already shifted but the seg is dropped (not appended), so no harm.

Let me now seriously evaluate the **padding overlap "replay" contract violation** and write it up as the main finding, plus check one more area ? the `window_inactivity_end` / blob fallback in the runner to see if there's an off-by-one. Let me check `trajectory_to_action_windows` quickly for the windowing math.

46. user (2026-06-18T07:50:52.153Z)

```
/c/Users/avidu/Projects/badminton-highlight-
indexer/backend/pipeline/detectors/tracknet_runner.py:256:    def trajectory_to_action_windows(
```

47. assistant (2026-06-18T07:50:56.980Z)

That's outside my assigned AREA (detectors), but it's called by the segmenter. Let me stay focused on segmenters/stitcher/validators. Let me do a final targeted check on the begin/end padding contract and confirm the "replay" overlap is real and reachable with default config. Let me verify default padding values and how the compiler is constructed.

48. user (2026-06-18T07:50:58.328Z)

```
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py:240:
begin_padding: float = 0.5
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py:241:
end_padding: float = 0.5
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py:620:    compiler =
VideoCompiler(output_dir, begin_padding=stitching.begin_padding,
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py:621:
end_padding=stitching.end_padding, watermark=stitching.watermark)
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py:755:        compiler =
VideoCompiler(out_dir, begin_padding=stitching.begin_padding,
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py:756:
end_padding=stitching.end_padding, watermark=stitching.watermark)
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\stitcher\compiler.py:17:
def __init__(self, output_dir: str, end_padding: float = 1.0, begin_padding: float = 0.5,
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\stitcher\compiler.py:20:
self.end_padding = end_padding
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\stitcher\compiler.py:21:
self.begin_padding = begin_padding
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\stitcher\compiler.py:193:
into a seamless highlights file, applying end_padding to prevent abrupt endings.
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\stitcher\compiler.py:228:
logger.info(f"Trimming {len(segments)} segments (begin_padding={self.begin_padding}s,
end_padding={self.end_padding}s)...")
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\stitcher\compiler.py:268:
padded_start = max(0.0, start - self.begin_padding)
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\stitcher\compiler.py:269:
padded_end = end + self.end_padding
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\stitcher\compiler.py:274:
logger.info(f"{segment_label}: Padded end ({end + self.end_padding:.2f}s) exceeds video
duration. "
```

```
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\segmenters\gemini.py:420:
begin_pad = stitch_cfg.get("begin_padding", 0.5)
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\segmenters\gemini.py:421:
end_pad = stitch_cfg.get("end_padding", 0.5)
```

49. assistant (2026-06-18T07:51:11.303Z)

Default padding is 0.5+0.5 = 1.0s. The "replay" overlap is real but small (?1s of duplicated footage between rallies closer than the combined padding) and is an inherent design tradeoff of independent per-clip padding. I'll report it as low severity/low-medium confidence since the module's own docstring asserts "must never replay the same footage."

Now let me reconsider whether there's a sharper, higher-confidence bug. Let me re-examine the **scene_cut validator `sample_interval` and the seek behavior** vs the cap-at-100 logic combined with `cuts_per_minute`. Actually, let me re-examine the relevance validator default green range against the documented "badminton court" ? irrelevant to bugs.

Let me reconsider `ResolutionFPSValidator` `r_frame_rate` parse: `num, den = map(int, fps_str.split("/"))`. If `fps_str` is "30" (no slash), `split("/")` gives ["30"], unpacking into 2 vars ? ValueError, caught ? fps=0.0 ? fails the FPS check. ffprobe always returns "N/N" format for r_frame_rate, so not reachable in practice. Low.

Let me look at the one genuinely suspicious unguarded access: in `relevance.py`, `frame.shape[:2]` (line 64) ? if `cap.read()` returns `ret=True` but a malformed frame... not realistic.

Let me verify the **`max(0, window["start"] - handoff_padding)` type issue** is benign and check the never-empty fusion guard once more, then also examine whether `handoff_indices` being empty in the NON-fusion base would cause issues. Base `_select_for_handoff` returns `list(range(len(windows)))` ? never empty if windows non-empty. The `if not all_candidate_windows: return []` guard is at line 246. So handoff loop only runs with ?1 window. Fine.

Let me look at the test files for the hybrid segmenters to confirm what's already covered and not waste a finding on tested behavior.

50. user (2026-06-18T07:51:11.850Z)

```
tests\test_fusion_compare.py
tests\test_fusion_features.py
tests\test_fusion_golden_telemetry.py
tests\test_fusion_hybrid.py
```

51. user (2026-06-18T07:51:12.311Z)

```
tests\test_trajectory_hybrid_equivalence.py
```

52. user (2026-06-18T07:51:12.328Z)

```
tests\test_validators_base.py
```

53. assistant (2026-06-18T07:51:15.478Z)

Let me check the validator and motion tests to see what's covered, particularly to confirm the motion segmenter's behavior isn't already guarded against the short-segment smash edge.

54. user (2026-06-18T07:51:16.018Z)

```
tests\test_promotion.py
```

55. user (2026-06-18T07:51:16.527Z)

```
1       """Tests for the validation registry, focused on the backward-compatible
2       `run_all_with_context` added for observability reporting."""
```

```
    from backend.pipeline.validators.base import ValidationRegistry, ValidationResult,
VideoValidator


    class _ProducerValidator(VideoValidator):
        @property
        def name(self):
            return "producer"

        @property
        def description(self):
            return "writes ffprobe_meta into the shared context"

        def validate(self, video_path, config, context):
            context["ffprobe_meta"] = {"fps": 59.94, "width": 1920, "height": 1080}
            return ValidationResult(validator_name=self.name, passed=True, message="ok")


    class _ConsumerValidator(VideoValidator):
        @property
        def name(self):
            return "consumer"

        @property
        def description(self):
            return "reads ffprobe_meta the producer cached"

        def validate(self, video_path, config, context):
            seen = "ffprobe_meta" in context
            return ValidationResult(validator_name=self.name, passed=seen,
message=f"saw_meta={seen}")


    def _registry():
        r = ValidationRegistry()
        r.register(_ProducerValidator())
        r.register(_ConsumerValidator())
        return r


    def test_run_all_with_context_returns_results_and_context():
        results, context = _registry().run_all_with_context("v.mp4", {})
        assert [r.validator_name for r in results] == ["producer", "consumer"]
        assert context["ffprobe_meta"]["fps"] == 59.94
        assert results[1].passed is True  # consumer saw the shared context


    def test_run_all_is_unchanged_and_returns_only_results():
        results = _registry().run_all("v.mp4", {})
        assert isinstance(results, list)
        assert all(isinstance(r, ValidationResult) for r in results)
        assert [r.validator_name for r in results] == ["producer", "consumer"]


    def test_validator_exception_is_captured_not_raised():
        class _Boom(VideoValidator):
            @property
            def name(self):
                return "boom"

            @property
            def description(self):
                return "raises"

            def validate(self, video_path, config, context):
```

```
                raise RuntimeError("explode")

        r = ValidationRegistry()
        r.register(_Boom())
        results, _ = r.run_all_with_context("v.mp4", {})
        assert results[0].passed is False
        assert "explode" in results[0].message


    def test_validator_registry_safe_with_complex_instance():
        """Documents that ValidationRegistry is instance-based and therefore has no Item-3
        discovery footgun.

        Unlike SegmenterRegistry (and the sports/annotations registries),
ValidationRegistry.register()
        accepts an already-constructed instance and simply appends it (see
backend/pipeline/validators/base.py:38-40).
        There is no `cls(None, {})` temp instantiation, no .name/.description discovery at
registration time,
        and no list_available() path. Therefore a validator's __init__ (even if
complex/expensive or
        side-effecting) can never be triggered by the registry itself.

        This test proves the (desirable) property that pre-built instances with heavy state
can be
        registered and run safely. It is retained for completeness and to explicitly
document the
        structural difference vs. the registries that *do* have the footgun. It would not
catch a
        regression that re-introduced discovery-style instantiation (because this path has
none).
        See PR #156 review feedback and CODE_AUDIT_AND_TEST_HARDENING.md for context.
        """
        class ComplexValidator(VideoValidator):
            def __init__(self):
                # Simulate complex state (e.g. heavy computation, resource acquisition)
                # that would be undesirable or error-prone if "discovered" repeatedly.
                self._heavy_state = list(range(10000))  # or real db/conn in future

            @property
            def name(self):
                return "complex-validator"

            @property
            def description(self):
                return "Tests safe instance registration for validators with complex
__init__ state."

            def validate(self, video_path, config, context):
                # Use the state to "prove" it was initialized once
                return ValidationResult(
                    validator_name=self.name,
                    passed=len(self._heavy_state) > 0,
                    message="ok"
                )

        r = ValidationRegistry()
        v = ComplexValidator()
        r.register(v)

        results, _ = r.run_all_with_context("v.mp4", {})
        assert len(results) == 1
        assert results[0].validator_name == "complex-validator"
        assert results[0].passed is True

```

56. assistant (2026-06-18T07:51:32.757Z)

I have now thoroughly reviewed the AREA. Let me do a final consideration of the most defensible findings and assign honest confidence. Let me re-examine one thing I want to be precise about: the watermark silent-empty-output fallback gap, and the padding overlap. Let me also reconsider the `ValidationResult` construction in `run_all_with_context` exception handler ? it builds `ValidationResult` without `details`, which defaults to `{}`. Fine.

Let me reconsider whether the **`backward_jumps` / B-frame PTS** in pts_continuity is a real false-positive bug. The validator extracts `packet=pts_time` for keyframes only (`if "K" in flags`). Keyframes (I-frames) always have monotonically increasing PTS ? B-frame reordering affects DTS vs PTS within a GOP but keyframe PTS_time values themselves increase. So `backward_jumps` shouldn't false-fire on normal video. But it filters to keyframes via `"K" in flags` ? ffprobe packet flags for a keyframe is "K_" or "K". For non-keyframes it's "__". So only I-frame PTS collected, monotonic. OK, no bug.

But wait ? there's a subtle real issue: `packet=pts_time` can be "N/A" for some packets. `float("N/A")` ? ValueError ? `continue` (line 47). Handled.

I'm now confident in my findings. Let me reconsider the strongest one ? the watermark silent-empty fallback ? and verify it's genuinely reachable. The retry only happens on `CalledProcessError`. ffmpeg with a bad `-vf` filter exits non-zero (CalledProcessError) in the vast majority of cases, so the retry DOES trigger normally. The "exit 0 + empty output" with a watermark is a rare ffmpeg edge. Low confidence ? I'll include it as low.

Let me settle on reporting the padding-overlap "replay" contract issue and a couple of genuinely-defensible lower-severity items, being honest about confidence. Let me do one last check: the fusion `id(points)` cache to decide if it's worth reporting.

The `_axis_cache` is keyed on `id(points)`. Within one `process_video`, points is one stable list ? fine. The risk is only across calls with address reuse, AND would only mis-set the deadband/net slightly. It's a known Python `id()`-as-cache-key antipattern but the practical blast radius is negligible and the same-instance-reuse-across-videos pattern isn't how segmenters are used (one instance per run typically). I'll report it as a low-confidence latent issue.

Let me finalize. I have enough to report precisely.

57. user (2026-06-18T07:51:51.578Z)

Output does not match required schema: /candidates/0: must NOT have additional properties

58. assistant (2026-06-18T07:51:54.391Z)

The first candidate has an extra `why_wrong_detail` property. Let me resubmit with the correct shape.

59. user (2026-06-18T07:52:07.430Z)

Structured output provided successfully

60. assistant (2026-06-18T07:52:16.826Z)

I reviewed the assigned AREA (segmenters, stitcher/compiler, validators) reading every file plus the matching tests and the DB/video helper contracts. The codebase is well-hardened; I found no crash-on-reachable-input, inverted condition, or unguarded persisted-data access. The merge/padding math, subprocess returncode handling, the never-empty fusion gate, candidate-index/handoff alignment, and the validators' context dependency are all sound.

I returned three low-severity, honestly-scored candidates (no high/medium-confidence defects):

1. compiler.py:268-269 ? the "never replay the same footage" invariant is enforced only on unpadded ranges, but begin/end padding is added per-clip afterward, so two rallies closer than the combined padding (default 1.0s) emit overlapping frames that play twice. Real contract

violation, small blast radius (cosmetic seam overlap). Confidence 0.5.

2. compiler.py:300-321 ? the watermark un-watermarked fallback only triggers on CalledProcessError; a watermark encode that exits 0 but produces an empty/missing clip skips the retry and drops the segment, contradicting the "branding misconfig can never nuke the reel" guarantee. Rare ffmpeg edge. Confidence 0.45.

3. fusion_hybrid.py:100-109 ? the play-axis cache is keyed on id(points), which is unsound across object lifetimes (freed-address reuse) and could return a stale net/axis estimate on a reused instance across videos, mis-scoring net_crossings. Safe within a single process_video. Confidence 0.3.